Technological Institute of the Philippines	Quezon City - Computer Engineering
Course Code:	CPE 312
Code Title:	Predictive Analytics using Machine Learning
1st Semester	AY 2026-2027
HOA 2.1	**Training Linear Models**
Name	Abon, Benedict Aldous
Section	CPE31S3
Date Performed:	7/23/26
Date Submitted:	7/23/26
Instructor:	Engr. Robin Valenzuela

1. Objectives

This activity aims to:

- Perform regression analysis using linear regression and polynomial regression; and
- Solve classification problem using logistic regression.

2. Intended Learning Outcomes (ILOs)

After this activity, the students should be able to:

- Demonstrate how to use Python to predic outcome using linear and polynomial regression;
- Demonstrate how to use single and multiple features to predict the outcome using linear regression;
- Demonstrate how to train and predict classification model using logistic regression;
- Evaluate and visualize the performance of different regression models.

3. Procedures and Outputs

3.1 Setup

These are configurations for running on your local machine. No problem with running it on your colab. This project requires Python 3.7 or above:

```
import sys

assert sys.version_info >= (3, 7)
```

It also requires Scikit-Learn ≥ 1.0.1:

```
from packaging import version
import sklearn
```

```
assert version.parse(sklearn.__version__) >= version.parse("1.0.1")
```

As we did in previous chapters, let's define the default font sizes to make the figures prettier:

```
import matplotlib.pyplot as plt

plt.rc('font', size=14)
plt.rc('axes', labelsize=14, titlesize=14)
plt.rc('legend', fontsize=14)
plt.rc('xtick', labelsize=10)
plt.rc('ytick', labelsize=10)
```

And let's create the `images/training_linear_models` folder (if it doesn't already exist), and define the `save_fig()` function which is used through this notebook to save the figures in high-res for the book:

```
from pathlib import Path

IMAGES_PATH = Path() / "images" / "training_linear_models"
IMAGES_PATH.mkdir(parents=True, exist_ok=True)

def save_fig(fig_id, tight_layout=True, fig_extension="png",
resolution=300):
    path = IMAGES_PATH / f"{fig_id}.{fig_extension}"
    if tight_layout:
        plt.tight_layout()
    plt.savefig(path, format=fig_extension, dpi=resolution)
```

3.2 Linear Regression

A linear model makes a prediction by simply computing a weighted sum of the input features, plus a constant called the *bias term* or the *intercept term*.

3.2.1 The Normal Equation

```
import numpy as np

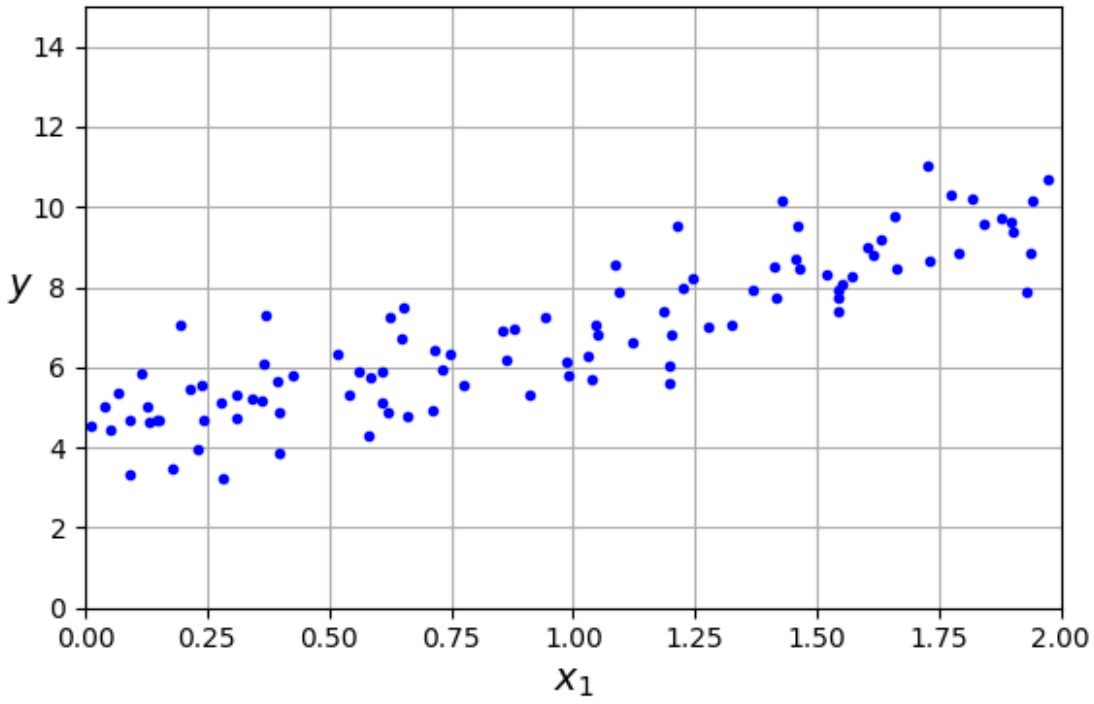
np.random.seed(42)  # to make this code example reproducible
m = 100  # number of instances
X = 2 * np.random.rand(m, 1)  # column vector
y = 4 + 3 * X + np.random.randn(m, 1)  # column vector

# extra code – generates and saves Figure 4–1

import matplotlib.pyplot as plt

plt.figure(figsize=(6, 4))
plt.plot(X, y, "b.")
plt.xlabel("$x_1$")
```

```
plt.ylabel("$y$", rotation=0)
plt.axis([0, 2, 0, 15])
plt.grid()
save_fig("generated_data_plot")
plt.show()
```




```
from sklearn.preprocessing import add_dummy_feature

X_b = add_dummy_feature(X)  # add x0 = 1 to each instance
theta_best = np.linalg.inv(X_b.T @ X_b) @ X_b.T @ y

theta_best
```

```
array([[4.21509616],
       [2.77011339]])
```

```
X_new = np.array([[0], [2]])
X_new_b = add_dummy_feature(X_new)  # add x0 = 1 to each instance
y_predict = X_new_b @ theta_best
y_predict
```

```
array([[4.21509616],
       [9.75532293]])
```

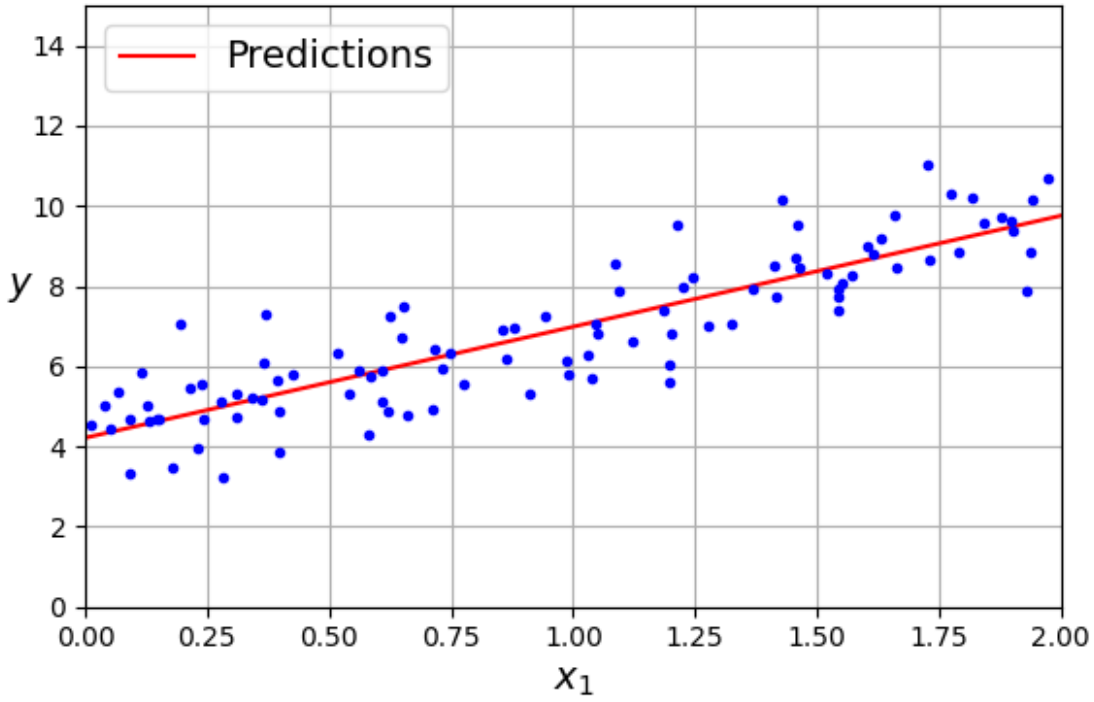
```
import matplotlib.pyplot as plt

plt.figure(figsize=(6, 4))  # extra code – not needed, just formatting
plt.plot(X_new, y_predict, "r-", label="Predictions")
```

```
plt.plot(X, y, "b.")

# extra code – beautifies and saves Figure 4–2
plt.xlabel("$x_1$")
plt.ylabel("$y$", rotation=0)
plt.axis([0, 2, 0, 15])
plt.grid()
plt.legend(loc="upper left")
save_fig("linear_model_predictions_plot")

plt.show()
```




```
from sklearn.linear_model import LinearRegression

lin_reg = LinearRegression()
lin_reg.fit(X, y)
lin_reg.intercept_, lin_reg.coef_

(array([4.21509616]), array([[2.77011339]]))

lin_reg.predict(X_new)

array([[4.21509616],
       [9.75532293]])
```

The `LinearRegression` class is based on the `scipy.linalg.lstsq()` function (the name stands for "least squares"), which you could call directly:

```
theta_best_svd, residuals, rank, s = np.linalg.lstsq(X_b, y, rcond=1e-6)
theta_best_svd
```

```
array([[4.21509616],
       [2.77011339]])
```

The pseudoinverse is computed using a standard matrix factorization technique called *Singular Value Decomposition*. What does SVD do? Quickly research and answer.

- According to IBM, singular value decomposition is defined as a way of breaking any matrix into three simpler matrices that reveal its underlying structure, an important tool in machine learning and data science.

This function computes $X^{+¿y¿}$, where $X^{+¿¿}$ is the *pseudoinverse* of X (specifically the Moore-Penrose inverse). You can use `np.linalg.pinv()` to compute the pseudoinverse directly:

```
np.linalg.pinv(X_b) @ y
```

```
array([[4.21509616],
       [2.77011339]])
```

You have multiple methods used to implement the for linear regression. Provide a comparison between the *Singular Value Decomposition* and the Normal Equation Computation. Which is more efficient?

- The Normal Equation Computation predicts by simply computes for the weighted sum of the input features plus the bias term, while the Singular Value Decomposition decomposes the data matrix to find the pseudoinverse. The SVD makes the better, efficient choice for getting the linear regression as it is more applicable to real data, makes dealing with noisy data easier.

3.2 Gradient Descent

Gradient Descent is a very generic optimization algorithm capable of finding optimal solutions to a wide range of problems. Generally, the idea is to tweak parameters iteratively to minimize a cost funtion.

3.2.1 Batch Gradient Descent

To implement Gradient Descent, you need to compute the gradient of the cost func tion with regards to each model parameter θj. In other words, you need to calculate how much the cost function will change if you change θj just a little bit. This is called a partial derivative. It is like asking “what is the slope of the mountain under my feet if I face east?” and then asking the same question facing north (and so on for all other dimensions, if you can imagine a universe with more than three dimensions).

```
eta = 0.1  # learning rate
n_epochs = 1000
m = len(X_b)  # number of instances
```

```
np.random.seed(42)
theta = np.random.randn(2, 1)  # randomly initialized model parameters

for epoch in range(n_epochs):
    gradients = 2 / m * X_b.T @ (X_b @ theta - y)
    theta = theta - eta * gradients
```

The trained model parameters:

```
theta
```

```
array([[4.21509616],
       [2.77011339]])
```

```
# extra code – generates and saves Figure 4–8

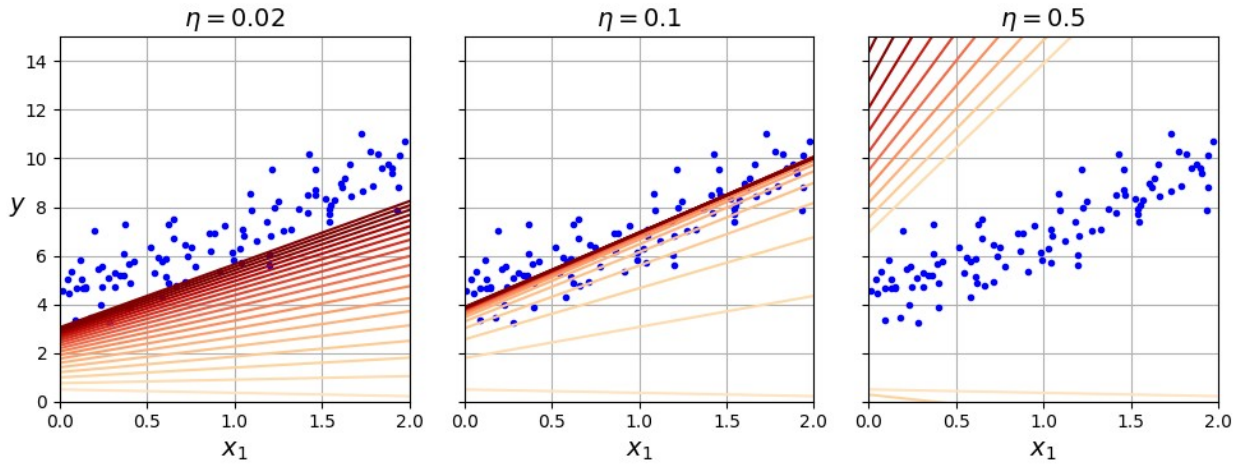
import matplotlib as mpl

def plot_gradient_descent(theta, eta):
    m = len(X_b)
    plt.plot(X, y, "b.")
    n_epochs = 1000
    n_shown = 20
    theta_path = []
    for epoch in range(n_epochs):
        if epoch < n_shown:
            y_predict = X_new_b @ theta
            color = mpl.colors.rgb2hex(plt.cm.OrRd(epoch / n_shown +
0.15))
            plt.plot(X_new, y_predict, linestyle="solid", color=color)
        gradients = 2 / m * X_b.T @ (X_b @ theta - y)
        theta = theta - eta * gradients
        theta_path.append(theta)
    plt.xlabel("$x_1$")
    plt.axis([0, 2, 0, 15])
    plt.grid()
    plt.title(fr"$\eta = {eta}$")
    return theta_path

np.random.seed(42)
theta = np.random.randn(2, 1)  # random initialization

plt.figure(figsize=(10, 4))
plt.subplot(131)
plot_gradient_descent(theta, eta=0.02)
plt.ylabel("$y$", rotation=0)
plt.subplot(132)
theta_path_bgd = plot_gradient_descent(theta, eta=0.1)
plt.gca().axes.yaxis.set_ticklabels([])
```

```
plt.subplot(133)
plt.gca().axes.yaxis.set_ticklabels([])
plot_gradient_descent(theta, eta=0.5)
save_fig("gradient_descent_plot")
plt.show()
```



Compare the learning rate provided and shown in the graph above. Provide a quick discussion on how learning rate affects model learning based on the given figure.

- Based on the created figure, the gradient descent is an iterative process of minimizing the model's error by gradually adjusting its parameters. In this example, we can see how the learning rate affects the model training process. Making the learning rate too high or low may lead to inaccurate results, otherwise, it will return an optimal model training result.

3.2.2 Stochastic Gradient Descent

The main problem with Batch Gradient Descent is the fact that it uses the whole training set to compute the gradients at every step, which makes it very slow when the training set is large. At the opposite extreme, Stochastic Gradient Descent just picks a random instance in the training set at every step and computes the gradients based only on that single instance. Obviously this makes the algorithm much faster since it has very little data to manipulate at every iteration. It also makes it possible to train on huge training sets, since only one instance needs to be in memory at each iteration (SGD can be implemented as an out-of-core algorithm.)

```
theta_path_sgd = []  # extra code – we need to store the path of theta
in the
                     #              parameter space to plot the next
figure

n_epochs = 50
t0, t1 = 5, 50  # learning schedule hyperparameters

def learning_schedule(t):
    return t0 / (t + t1)
```

```
np.random.seed(42)
theta = np.random.randn(2, 1)  # random initialization

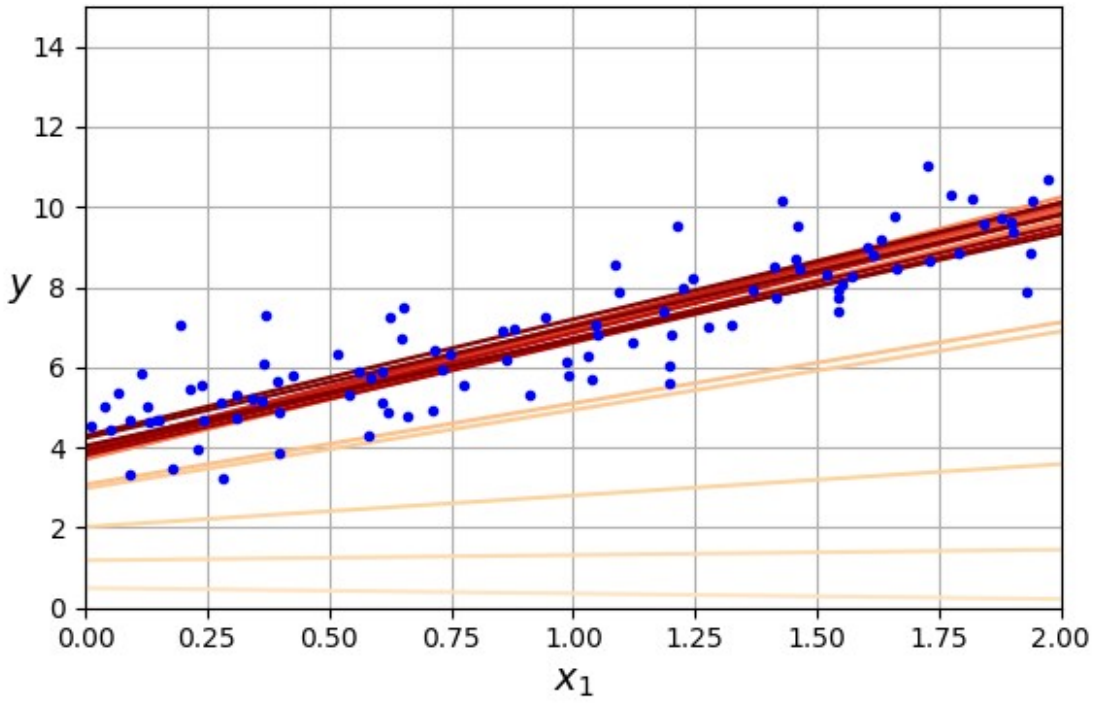
n_shown = 20  # extra code – just needed to generate the figure below
plt.figure(figsize=(6, 4))  # extra code – not needed, just formatting

for epoch in range(n_epochs):
    for iteration in range(m):

        # extra code – these 4 lines are used to generate the figure
        if epoch == 0 and iteration < n_shown:
            y_predict = X_new_b @ theta
            color = mpl.colors.rgb2hex(plt.cm.OrRd(iteration / n_shown
+ 0.15))
            plt.plot(X_new, y_predict, color=color)

        random_index = np.random.randint(m)
        xi = X_b[random_index : random_index + 1]
        yi = y[random_index : random_index + 1]
        gradients = 2 * xi.T @ (xi @ theta - yi)  # for SGD, do not
divide by m
        eta = learning_schedule(epoch * m + iteration)
        theta = theta - eta * gradients
        theta_path_sgd.append(theta)  # extra code – to generate the
figure

# extra code – this section beautifies and saves Figure 4–10
plt.plot(X, y, "b.")
plt.xlabel("$x_1$")
plt.ylabel("$y$", rotation=0)
plt.axis([0, 2, 0, 15])
plt.grid()
save_fig("sgd_plot")
plt.show()
```




```
theta
```

```
array([[4.21076011],
       [2.74856079]])
```

```
from sklearn.linear_model import SGDRegressor

sgd_reg = SGDRegressor(max_iter=1000, tol=1e-5, penalty=None,
eta0=0.01,
                       n_iter_no_change=100, random_state=42)
sgd_reg.fit(X, y.ravel())  # y.ravel() because fit() expects 1D
targets
```

```
SGDRegressor(n_iter_no_change=100, penalty=None, random_state=42,
tol=1e-05)
```

```
sgd_reg.intercept_, sgd_reg.coef_
```

```
(array([4.21278812]), array([2.77270267]))
```

Compare the Stochastic Gradient Descent used in this section to the gradient descent algorithm used previously. What are the benefits to using Stochastic Gradient Descent (SGD)?

- aaa
- Stochastic Gradient Descent updates its parameters as it iterate through every single random data point, while Batch Gradient Descent uses the whole dataset to update its parameter. SGD updates faster than the other and can handle massive datasets, while BGD is more stable and can create more accurate results.

3.2.3 Mini-batch gradient descent

The last Gradient Descent algorithm we will look at is called Mini-batch Gradient Descent. It is quite simple to understand once you know Batch and Stochastic Gradient Descent: at each step, instead of computing the gradients based on the full training set (as in Batch GD) or based on just one instance (as in Stochastic GD), Minibatch GD computes the gradients on small random sets of instances called minibatches. The main advantage of Mini-batch GD over Stochastic GD is that you can get a performance boost from hardware optimization of matrix operations, especially when using GPUs.

```
# extra code – this cell generates and saves Figure 4–11

from math import ceil

n_epochs = 50
minibatch_size = 20
n_batches_per_epoch = ceil(m / minibatch_size)

np.random.seed(42)
theta = np.random.randn(2, 1)  # random initialization

t0, t1 = 200, 1000  # learning schedule hyperparameters

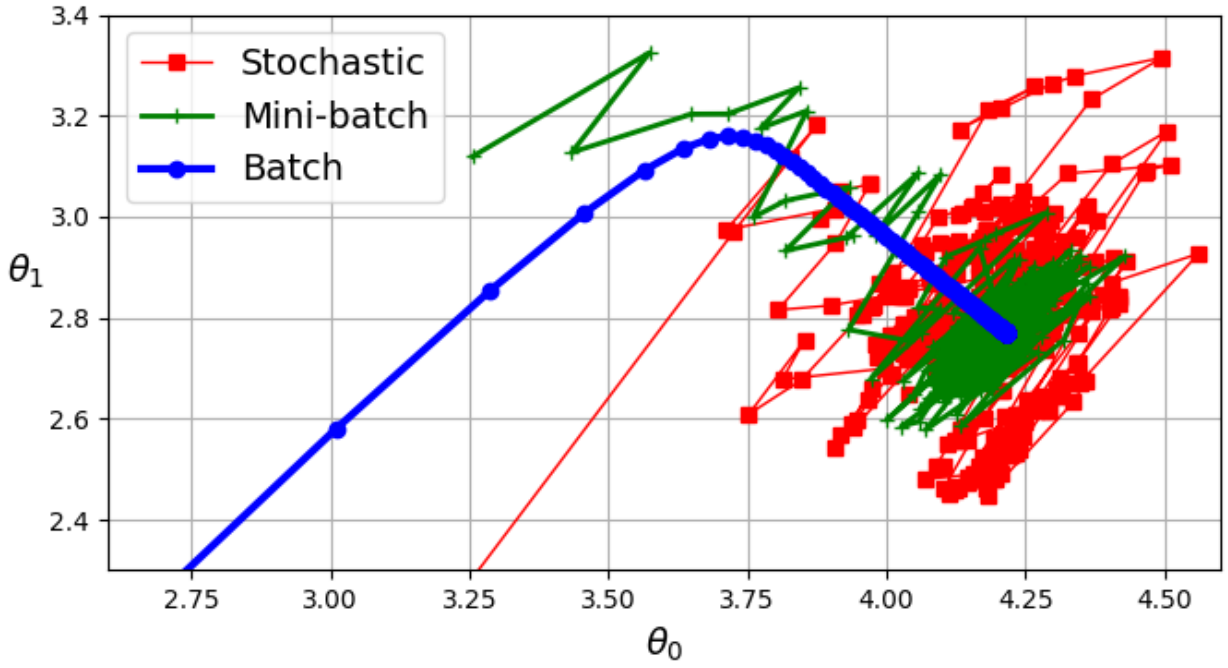
def learning_schedule(t):
    return t0 / (t + t1)

theta_path_mgd = []
for epoch in range(n_epochs):
    shuffled_indices = np.random.permutation(m)
    X_b_shuffled = X_b[shuffled_indices]
    y_shuffled = y[shuffled_indices]
    for iteration in range(0, n_batches_per_epoch):
        idx = iteration * minibatch_size
        xi = X_b_shuffled[idx : idx + minibatch_size]
        yi = y_shuffled[idx : idx + minibatch_size]
        gradients = 2 / minibatch_size * xi.T @ (xi @ theta - yi)
        eta = learning_schedule(iteration)
        theta = theta - eta * gradients
        theta_path_mgd.append(theta)

theta_path_bgd = np.array(theta_path_bgd)
theta_path_sgd = np.array(theta_path_sgd)
theta_path_mgd = np.array(theta_path_mgd)

plt.figure(figsize=(7, 4))
plt.plot(theta_path_sgd[:, 0], theta_path_sgd[:, 1], "r-s",
linewidth=1,
         label="Stochastic")
plt.plot(theta_path_mgd[:, 0], theta_path_mgd[:, 1], "g-+",
linewidth=2,
```

```
         label="Mini-batch")
plt.plot(theta_path_bgd[:, 0], theta_path_bgd[:, 1], "b-o",
linewidth=3,
         label="Batch")
plt.legend(loc="upper left")
plt.xlabel(r"$\theta_0$")
plt.ylabel(r"$\theta_1$    ", rotation=0)
plt.axis([2.6, 4.6, 2.3, 3.4])
plt.grid()
save_fig("gradient_descent_paths_plot")
plt.show()
```



The figure above, generated from the code in the cell `vXWApBO_8T3U`, visualizes the optimization paths of three different Gradient Descent algorithms in the parameter space:

- **Batch Gradient Descent (Batch):** Represented by the blue line with circles (`b-o`).
- **Stochastic Gradient Descent (Stochastic):** Represented by the red line with squares (`r-s`).
- **Mini-batch Gradient Descent (Mini-batch):** Represented by the green line with plus signs (`g-+`).

What `θ0` and `θ1` represent:

In the context of the linear regression model `y = θ0 + θ1 * x1` (or `y = theta[0] + theta[1] * X`), which was used to generate the data earlier in the notebook:

- **`θ0` (theta 0):** This represents the **intercept** of the linear regression line. It's the value of `y` when `x1` (the feature) is 0. It's plotted on the x-axis of this figure.

- **`θ1` (theta 1):** This represents the **slope** of the linear regression line. It's how much `y` changes for every unit increase in `x1`. It's plotted on the y-axis of this figure.

Each point on these colored paths corresponds to a pair of `(θ0, θ1)` values that the respective Gradient Descent algorithm iteratively found as it tried to minimize the cost function. The goal of all these algorithms is to reach the optimal `(θ0, θ1)` values that best fit the data (which are approximately `(4.21, 2.77)` as seen from the Normal Equation and `LinearRegression` results).

Meaning of the paths:

- **Batch Gradient Descent** takes a very direct, smooth path towards the optimal parameters because it calculates the gradient using the *entire* dataset in each step. This makes its convergence stable.
- **Stochastic Gradient Descent** shows a much more erratic and noisy path. This is because it updates the parameters based on the gradient of *just one random instance* at a time. While noisy, this can help it escape local minima and is faster for large datasets.
- **Mini-batch Gradient Descent** (which is not present in the graph currently, as it was commented out in `vXWApBO_8T3U`) would typically show a path that is less noisy than SGD but more noisy than BGD, as it uses a small *batch* of instances for each gradient calculation.

Provide a comparison of the 3 gradient descent algorithms included in this section. The comparison must provide an analysis based on different factors such as execution time (among others).

- The figure above shows how each Gradient Descent methods can affect the calculation. The BGD takes a very direct and smooth calculation as it uses the whole dataset in one go. The SGD shows a noisy path as it updates per data point. The Mini-batch GD show a less noisy path compared to SGD as it calculates the dataset by using small batches.

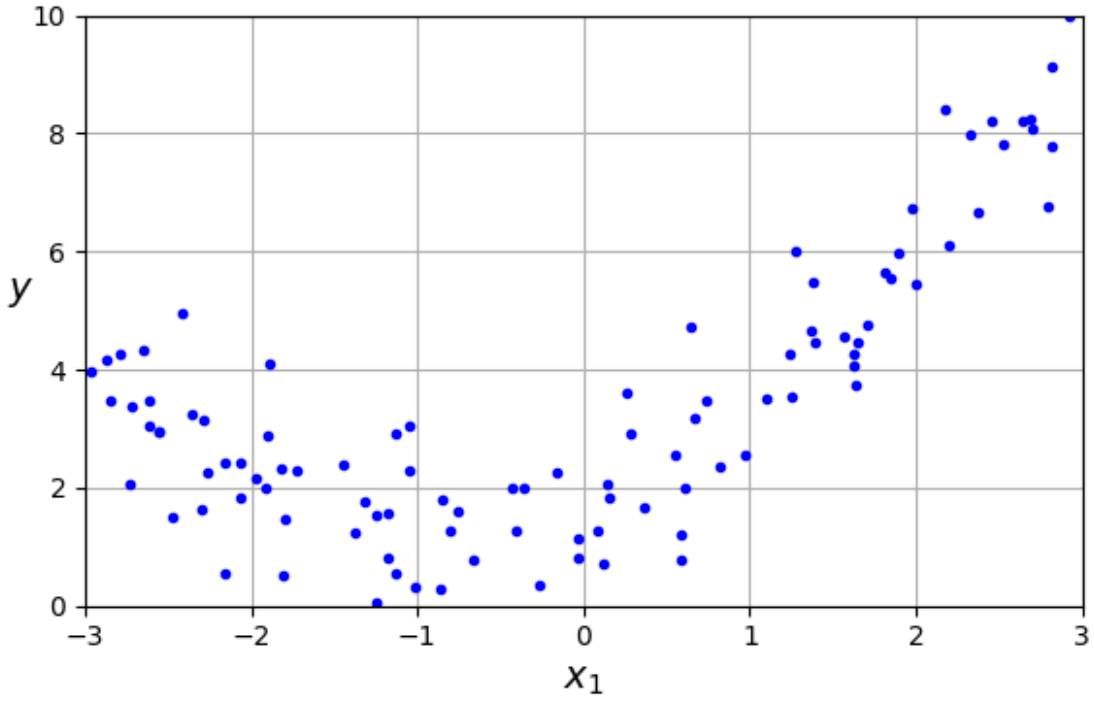
3.3 Polynomial Regression

What if your data is actually more complex than a simple straight line? Surprisingly, you can actually use a linear model to fit nonlinear data. A simple way to do this is to add powers of each feature as new features, then train a linear model on this extended set of features. This technique is called Polynomial Regression.

```
np.random.seed(42)
m = 100
X = 6 * np.random.rand(m, 1) - 3
y = 0.5 * X ** 2 + X + 2 + np.random.randn(m, 1)

# extra code – this cell generates and saves Figure 4–12
plt.figure(figsize=(6, 4))
plt.plot(X, y, "b.")
plt.xlabel("$x_1$")
plt.ylabel("$y$", rotation=0)
plt.axis([-3, 3, 0, 10])
plt.grid()
```

```
save_fig("quadratic_data_plot")
plt.show()
```




```
from sklearn.preprocessing import PolynomialFeatures

poly_features = PolynomialFeatures(degree=2, include_bias=False)
X_poly = poly_features.fit_transform(X)
X[0]
```

```
array([-0.75275929])
```

```
X_poly[0]
```

```
array([-0.75275929,  0.56664654])
```

```
lin_reg = LinearRegression()
lin_reg.fit(X_poly, y)
lin_reg.intercept_, lin_reg.coef_
```

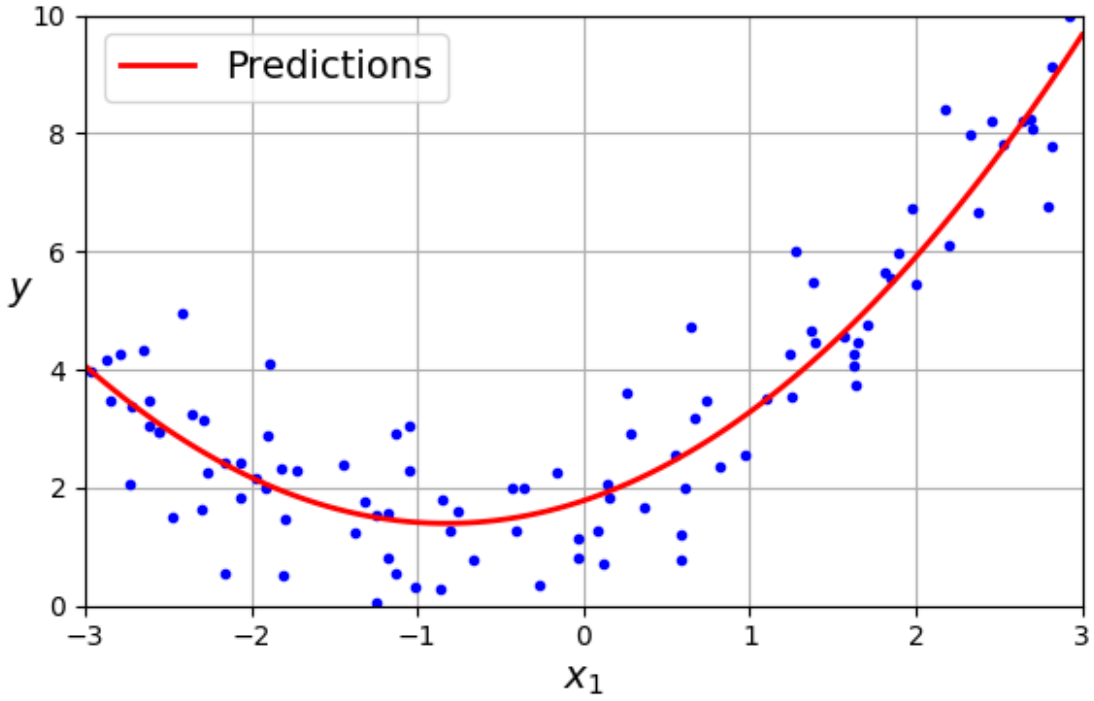
```
(array([1.78134581]), array([[0.93366893, 0.56456263]]))
```

```
# extra code – this cell generates and saves Figure 4–13

X_new = np.linspace(-3, 3, 100).reshape(100, 1)
X_new_poly = poly_features.transform(X_new)
y_new = lin_reg.predict(X_new_poly)

plt.figure(figsize=(6, 4))
```

```
plt.plot(X, y, "b.")
plt.plot(X_new, y_new, "r-", linewidth=2, label="Predictions")
plt.xlabel("$x_1$")
plt.ylabel("$y$", rotation=0)
plt.legend(loc="upper left")
plt.axis([-3, 3, 0, 10])
plt.grid()
save_fig("quadratic_predictions_plot")
plt.show()
```




```
# extra code – this cell generates and saves Figure 4–14

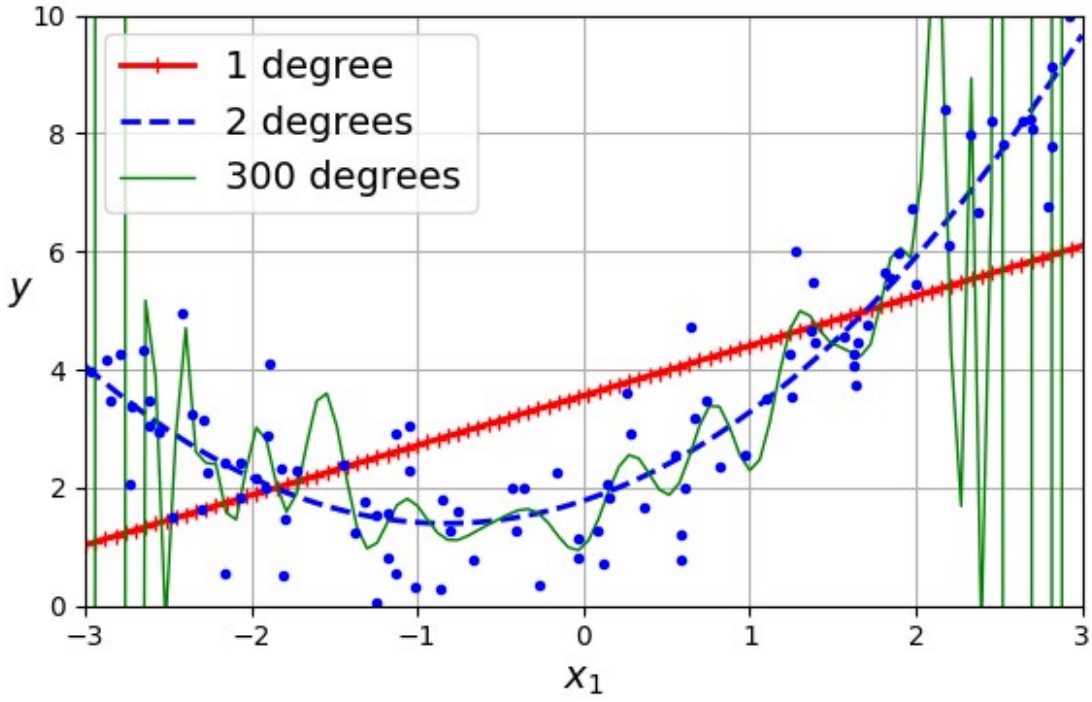
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

plt.figure(figsize=(6, 4))

for style, width, degree in (("r-+", 2, 1), ("b--", 2, 2), ("g-", 1,
300)):
    polybig_features = PolynomialFeatures(degree=degree,
include_bias=False)
    std_scaler = StandardScaler()
    lin_reg = LinearRegression()
    polynomial_regression = make_pipeline(polybig_features,
std_scaler, lin_reg)
    polynomial_regression.fit(X, y)
    y_newbig = polynomial_regression.predict(X_new)
```

```
    label = f"{degree} degree{'s' if degree > 1 else ''}"
    plt.plot(X_new, y_newbig, style, label=label, linewidth=width)

plt.plot(X, y, "b.", linewidth=3)
plt.legend(loc="upper left")
plt.xlabel("$x_1$")
plt.ylabel("$y$", rotation=0)
plt.axis([-3, 3, 0, 10])
plt.grid()
save_fig("high_degree_polynomials_plot")
plt.show()
```



Based on this example, how is Polynomial Regression able to find relationships between features? Is this something simple Linear Regression is unable to do?

What do we achieve in the figure above? Explain the result of the comparison.

- The Polynomial Regression find these relationships by creating combinations of these features and find their direction and strength of their relationship. This is something the Linear Regression can also do, but using the Polynomial Regression is more flexible. By changing the degrees depending on the dataset, we are able to accurately predict the dataset, which is why adding high degrees can lead to a noisy and overfitting prediction.

3.4 Learning Curves

If you perform high-degree Polynomial Regression, you will likely fit the training data much better than with plain Linear Regression.

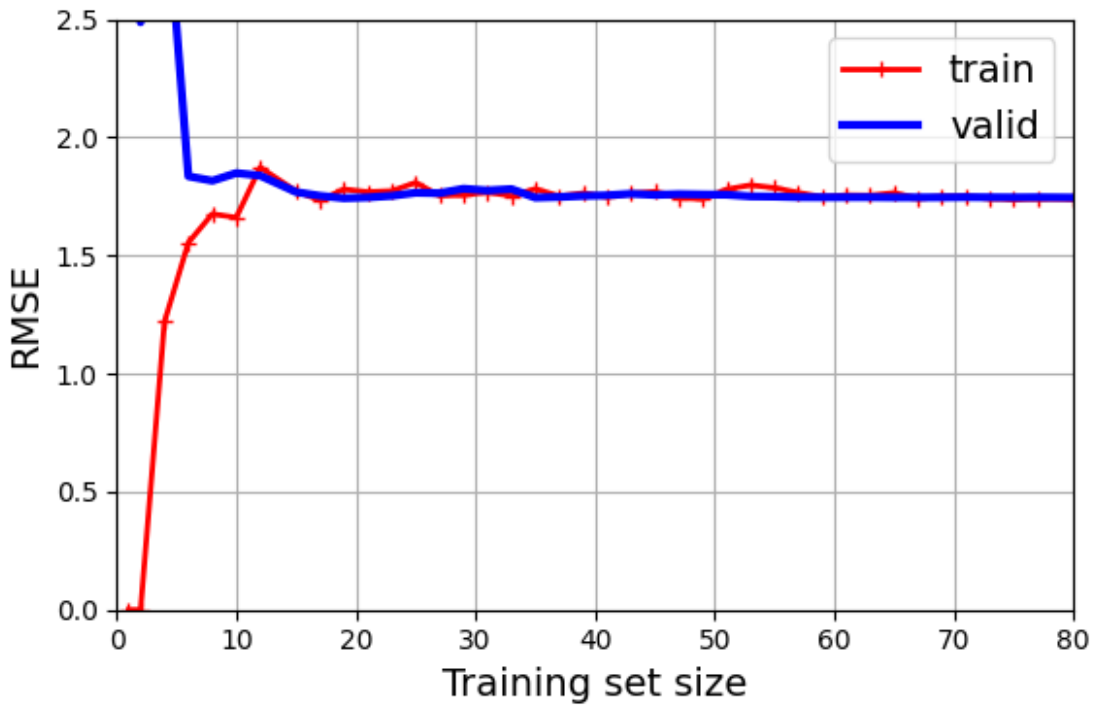
```
from sklearn.model_selection import learning_curve

train_sizes, train_scores, valid_scores = learning_curve(
    LinearRegression(), X, y, train_sizes=np.linspace(0.01, 1.0, 40),
cv=5,
    scoring="neg_root_mean_squared_error")
train_errors = -train_scores.mean(axis=1)
valid_errors = -valid_scores.mean(axis=1)

plt.figure(figsize=(6, 4))  # extra code – not needed, just formatting
plt.plot(train_sizes, train_errors, "r-+", linewidth=2, label="train")
plt.plot(train_sizes, valid_errors, "b-", linewidth=3, label="valid")

# extra code – beautifies and saves Figure 4–15
plt.xlabel("Training set size")
plt.ylabel("RMSE")
plt.grid()
plt.legend(loc="upper right")
plt.axis([0, 80, 0, 2.5])
save_fig("underfitting_learning_curves_plot")

plt.show()
```



This deserves a bit of explanation. First, let's look at the performance on the training data: when there are just one or two instances in the training set, the model can fit them perfectly, which is why the curve starts at zero. But as new instances are added to the training set, it becomes impossible for the model to fit the training data perfectly, both because the data is noisy and because it is not linear at all. So the error on the training data goes up until it reaches a plateau,

at which point adding new instances to the training set doesn't make the average error much better or worse.

Now let's look at the performance of the model on the validation data. When the model is trained on very few training instances, it is incapable of generalizing properly, which is why the validation error is initially quite big. Then as the model is shown more training examples, it learns and thus the validation error slowly goes down. However, once again a straight line cannot do a good job modeling the data, so the error ends up at a plateau, very close to the other curve.

What are these learning curves indicating? Is the model overfitting or underfitting? Explain why.

- Based on the figure, the model is overfitting as it has a low training error and high validation error. The training line is very low and consistently stays near zero as the training set size increases while the validation line shows higher levels that the training line.

```
from sklearn.pipeline import make_pipeline

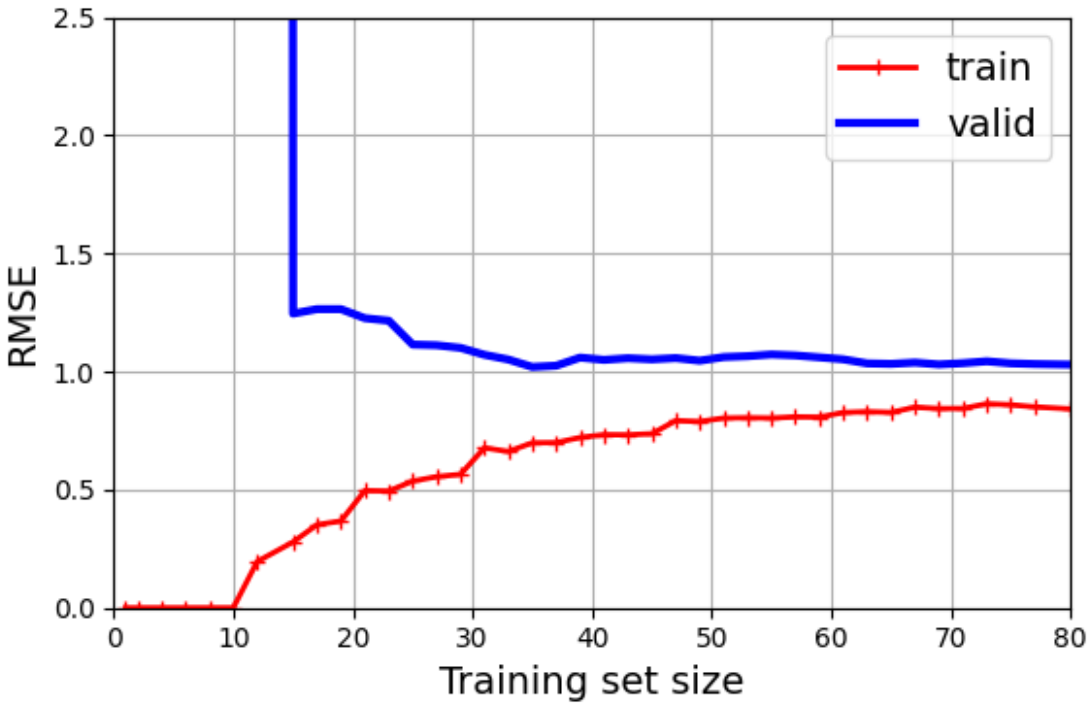
polynomial_regression = make_pipeline(
    PolynomialFeatures(degree=10, include_bias=False),
    LinearRegression())

train_sizes, train_scores, valid_scores = learning_curve(
    polynomial_regression, X, y, train_sizes=np.linspace(0.01, 1.0,
40), cv=5,
    scoring="neg_root_mean_squared_error")

# extra code – generates and saves Figure 4–16

train_errors = -train_scores.mean(axis=1)
valid_errors = -valid_scores.mean(axis=1)

plt.figure(figsize=(6, 4))
plt.plot(train_sizes, train_errors, "r-+", linewidth=2, label="train")
plt.plot(train_sizes, valid_errors, "b-", linewidth=3, label="valid")
plt.legend(loc="upper right")
plt.xlabel("Training set size")
plt.ylabel("RMSE")
plt.grid()
plt.axis([0, 80, 0, 2.5])
save_fig("learning_curves_plot")
plt.show()
```



Compare this learning curve with the one previously shown above. The major differences you must explain here are as follows:

- The learning rate
- The gap between the curves

Let's compare the two learning curves:

1. First Learning Curve (from cell `GMPzLig88T3X` - Simple Linear Regression)

- **Learning Rate (Error Convergence):** In this context, 'learning rate' refers to how quickly the model's error converges as more training data is provided. For the simple linear regression model, both the training error and the validation error quickly converge to a relatively high plateau. This indicates that even with more data, the model isn't significantly improving its ability to fit the data, nor generalize well.
- **Gap between the curves:** The gap between the training error (red line) and the validation error (blue line) is small. Both errors are high and converge to similar values. This pattern is characteristic of an **underfitting** model. The model is too simple (high bias) to capture the underlying patterns in the data, leading to poor performance on both the training and validation sets.

2. Second Learning Curve (from cell `2DnTAFTb8T3X` - Polynomial Regression with degree 10)

- **Learning Rate (Error Convergence):** The training error (red line) starts very low and stays low, indicating the model learns the training data almost perfectly. The validation error (blue line) starts very high when the training set is small, but then decreases and plateaus at a higher value than the training error. This suggests that the model can learn complex patterns but struggles to generalize them initially.

- **Gap between the curves:** There is a **large gap** between the training error (very low) and the validation error (significantly higher). This is the hallmark of an **overfitting** model. The high-degree polynomial model is too complex (high variance) and has essentially memorized the training data, including noise, rather than learning generalized patterns. As a result, it performs excellently on the training set but poorly on unseen validation data.
- The first learning curve has an error convergence, since the training and validation line converges very quickly, signifying that it does not fit and generalize the data very well. The second learning curve shows that the training line stays low and the validation line stars high and decreases, showing that the model learns complex patterns but lack generalization.

3.5 Regularized Linear Models

a good way to reduce overfitting is to regularize the model (i.e., to constrain it): the fewer degrees of freedom it has, the harder it will be for it to overfit the data. For example, a simple way to regularize a polynomial model is to reduce the number of polynomial degrees.

3.5.1 Ridge Regression

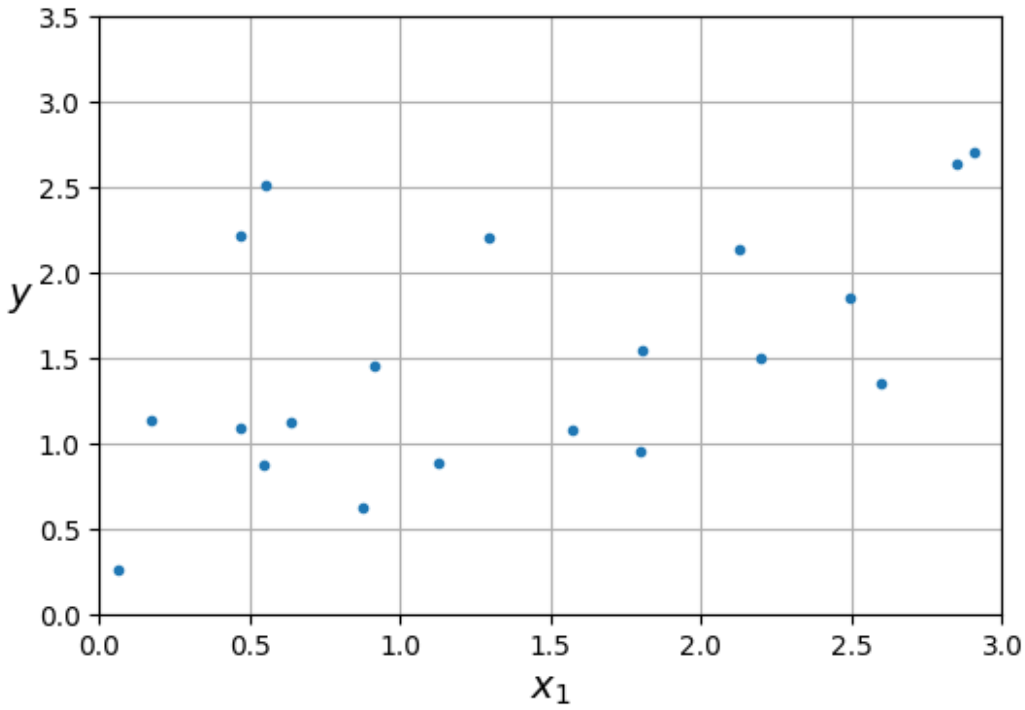
For a linear model, regularization is typically achieved by constraining the weights of the model. We will now look at Ridge Regression, Lasso Regression, and Elastic Net, which implement three different ways to constrain the weights.

This forces the learning algorithm to not only fit the data but also keep the model weights as small as possible. Note that the regularization term should only be added to the cost function during training. Once the model is trained, you want to evaluate the model's performance using the unregularized performance measure.

Let's generate a very small and noisy linear dataset:

```
# extra code – we've done this type of generation several times before
np.random.seed(42)
m = 20
X = 3 * np.random.rand(m, 1)
y = 1 + 0.5 * X + np.random.randn(m, 1) / 1.5
X_new = np.linspace(0, 3, 100).reshape(100, 1)

# extra code – a quick peek at the dataset we just generated
plt.figure(figsize=(6, 4))
plt.plot(X, y, ".")
plt.xlabel("$x_1$")
plt.ylabel("$y$  ", rotation=0)
plt.axis([0, 3, 0, 3.5])
plt.grid()
plt.show()
```




```
from sklearn.linear_model import Ridge

ridge_reg = Ridge(alpha=0.1, solver="cholesky")
ridge_reg.fit(X, y)
ridge_reg.predict([[1.5]])
```

```
array([1.55325833])
```

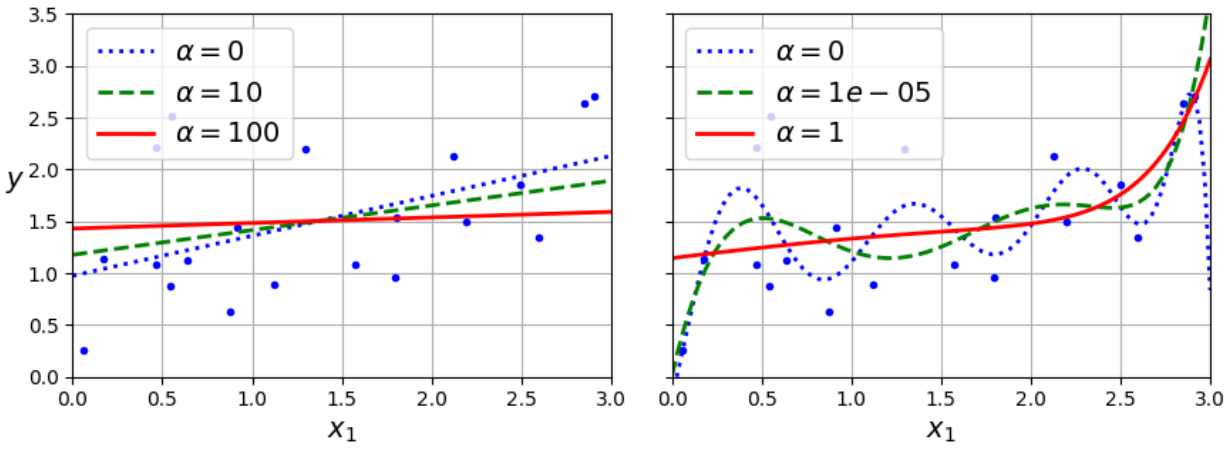
As with Linear Regression, we can perform Ridge Regression either by computing a closed-form equation or by performing Gradient Descent. The pros and cons are the same.

```
# extra code – this cell generates and saves Figure 4–17

def plot_model(model_class, polynomial, alphas, **model_kwargs):
    plt.plot(X, y, "b.", linewidth=3)
    for alpha, style in zip(alphas, ("b:", "g--", "r-")):
        if alpha > 0:
            model = model_class(alpha, **model_kwargs)
        else:
            model = LinearRegression()
        if polynomial:
            model = make_pipeline(
                PolynomialFeatures(degree=10, include_bias=False),
                StandardScaler(),
                model)
        model.fit(X, y)
        y_new_regul = model.predict(X_new)
```

```
        plt.plot(X_new, y_new_regul, style, linewidth=2,
                 label=fr"$\alpha = {alpha}$")
    plt.legend(loc="upper left")
    plt.xlabel("$x_1$")
    plt.axis([0, 3, 0, 3.5])
    plt.grid()

plt.figure(figsize=(9, 3.5))
plt.subplot(121)
plot_model(Ridge, polynomial=False, alphas=(0, 10, 100),
random_state=42)
plt.ylabel("$y$  ", rotation=0)
plt.subplot(122)
plot_model(Ridge, polynomial=True, alphas=(0, 10**-5, 1),
random_state=42)
plt.gca().axes.yaxis.set_ticklabels([])
save_fig("ridge_regression_plot")
plt.show()
```




```
sgd_reg = SGDRegressor(penalty="l2", alpha=0.1 / m, tol=None,
                       max_iter=1000, eta0=0.01, random_state=42)
sgd_reg.fit(X, y.ravel())  # y.ravel() because fit() expects 1D
targets
sgd_reg.predict([[1.5]])
```

```
array([1.55302613])
```

```
# extra code – show that we get roughly the same solution as earlier
when
#              we use Stochastic Average GD (solver="sag")
ridge_reg = Ridge(alpha=0.1, solver="sag", random_state=42)
ridge_reg.fit(X, y)
ridge_reg.predict([[1.5]])
```

```
array([1.55326019])
```

```
# extra code – shows the closed form solution of Ridge regression,
#              compare with the next Ridge model's learned parameters
below
alpha = 0.1
A = np.array([[0., 0.], [0., 1.]])
X_b = np.c_[np.ones(m), X]
np.linalg.inv(X_b.T @ X_b + alpha * A) @ X_b.T @ y
```

```
array([[0.97898394],
       [0.3828496 ]])
```

```
ridge_reg.intercept_, ridge_reg.coef_  # extra code
```

```
(array([0.97896386]), array([0.38286422]))
```

Discuss the following:

- What is the purpose of the hyperparameter alpha as seen in the figures above?
- Suppose you are using Ridge Regression and you notice that the training error and the validation error are almost equal and fairly high. Would you say that the model suffers from high bias or high variance? Should you increase the regularization hyperparameter α or reduce it?

1. Purpose of the hyperparameter `alpha` in Ridge Regression:

As seen in the `ridge_regression_plot` (Figure 4–17), `alpha` is a regularization hyperparameter in Ridge Regression. Its purpose is to control the amount of regularization applied to the model. Regularization aims to prevent overfitting by adding a penalty term to the cost function, which discourages the model from having large weights.

- **Higher `alpha` values:** Increase the regularization strength. This means the model is more constrained, and the weights are forced to be smaller. Graphically, as `alpha` increases (e.g., from 0 to 100 in the left plot, or 10^-5 to 1 in the right plot of Figure 4–17), the regression line becomes flatter and simpler. This reduces the model's complexity and its tendency to overfit, potentially leading to higher bias but lower variance.
- **Lower `alpha` values (closer to 0):** Reduce the regularization strength. When `alpha` is 0, Ridge Regression simply becomes equivalent to standard Linear Regression. The model is less constrained and can fit the training data more closely. This can lead to lower bias but potentially higher variance (overfitting).

In essence, `alpha` provides a trade-off between fitting the training data well (low bias) and keeping the model simple to generalize well to new data (low variance).

2. Scenario: Training error and validation error are almost equal and fairly high:

This scenario indicates that the model suffers from **high bias** (underfitting).

- **Why high bias?** When both training and validation errors are high and similar, it suggests that the model is too simple to capture the underlying patterns in the data. It's not performing well even on the data it was trained on, and therefore, it cannot generalize effectively to new data either. Both the training and validation errors

plateau at a high level, indicating the model lacks the complexity to learn from the data.

- **Should you increase or reduce `alpha`?** In this case, you should **reduce** the regularization hyperparameter `alpha`. A high `alpha` value would make the model even simpler (more constrained), which would worsen the underfitting problem. By reducing `alpha`, you allow the model to become more complex, potentially decreasing the bias and allowing it to fit the training data better, which should also lead to a decrease in validation error if the underfitting was the primary issue. You might also consider using a more complex model (e.g., higher polynomial degree if it's polynomial regression) or adding more features.

- The alpha hyperparameter controls the amount of regularization applied to the model, preventing overfitting. It adds a penalty term to the cost function, keeping the model weights balanced.
- The created model above shows high bias or underfitting. The model was too simple and does not capture too much of the patterns in it. With this, the alpha parameter must be reduced to add variance to the model, making the prediction more accurate and balanced.

3.5.2 Lasso Regression

Least Absolute Shrinkage and Selection Operator Regression (simply called Lasso Regression) is another regularized version of Linear Regression: just like Ridge Regression, it adds a regularization term to the cost function, but it uses the ℓ1 norm of the weight vector instead of half the square of the ℓ2 norm.

```
from sklearn.linear_model import Lasso

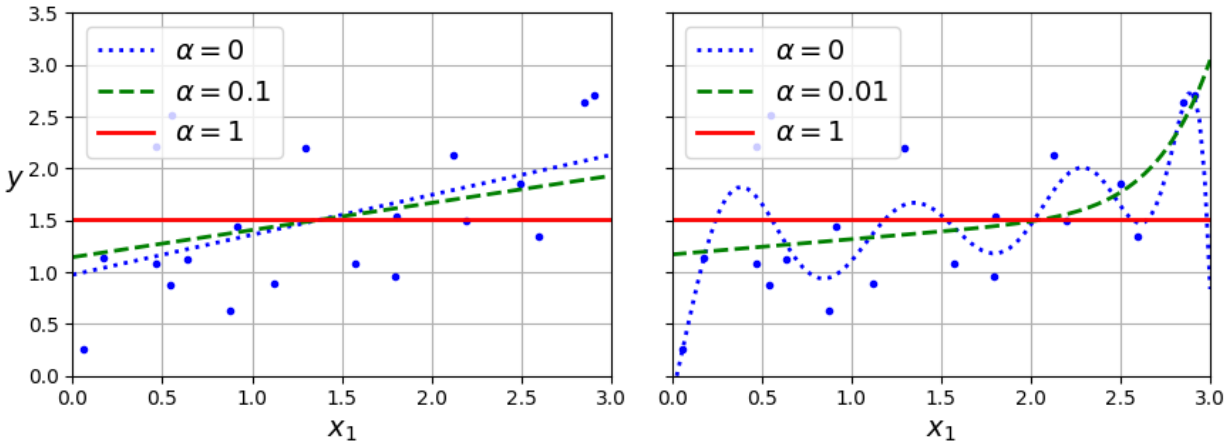
lasso_reg = Lasso(alpha=0.1)
lasso_reg.fit(X, y)
lasso_reg.predict([[1.5]])
```

```
array([1.53788174])
```

An important characteristic of Lasso Regression is that it tends to completely eliminate the weights of the least important features (i.e., set them to zero).

```
# extra code – this cell generates and saves Figure 4–18
plt.figure(figsize=(9, 3.5))
plt.subplot(121)
plot_model(Lasso, polynomial=False, alphas=(0, 0.1, 1),
random_state=42)
plt.ylabel("$y$  ", rotation=0)
plt.subplot(122)
plot_model(Lasso, polynomial=True, alphas=(0, 1e-2, 1),
random_state=42)
plt.gca().axes.yaxis.set_ticklabels([])
```

```
save_fig("lasso_regression_plot")
plt.show()
```



For example, the dashed line in the right plot on Figure 4-18 (with α = 10-7) looks quadratic, almost linear: all the weights for the high-degree polynomial features are equal to zero. In other words, Lasso Regression automatically performs feature selection and outputs a sparse model (i.e., with few nonzero feature weights).

```
# extra code – this BIG cell generates and saves Figure 4–19

t1a, t1b, t2a, t2b = -1, 3, -1.5, 1.5

t1s = np.linspace(t1a, t1b, 500)
t2s = np.linspace(t2a, t2b, 500)
t1, t2 = np.meshgrid(t1s, t2s)
T = np.c_[t1.ravel(), t2.ravel()]
Xr = np.array([[1, 1], [1, -1], [1, 0.5]])
yr = 2 * Xr[:, :1] + 0.5 * Xr[:, 1:]

J = (1 / len(Xr) * ((T @ Xr.T - yr.T) **
2).sum(axis=1)).reshape(t1.shape)

N1 = np.linalg.norm(T, ord=1, axis=1).reshape(t1.shape)
N2 = np.linalg.norm(T, ord=2, axis=1).reshape(t1.shape)

t_min_idx = np.unravel_index(J.argmin(), J.shape)
t1_min, t2_min = t1[t_min_idx], t2[t_min_idx]

t_init = np.array([[0.25], [-1]])

def bgd_path(theta, X, y, l1, l2, core=1, eta=0.05, n_iterations=200):
    path = [theta]
    for iteration in range(n_iterations):
        gradients = (core * 2 / len(X) * X.T @ (X @ theta - y)
                     + l1 * np.sign(theta) + l2 * theta)
```

```
        theta = theta - eta * gradients
        path.append(theta)
    return np.array(path)

fig, axes = plt.subplots(2, 2, sharex=True, sharey=True,
figsize=(10.1, 8))

for i, N, l1, l2, title in ((0, N1, 2.0, 0, "Lasso"), (1, N2, 0, 2.0,
"Ridge")):
    JR = J + l1 * N1 + l2 * 0.5 * N2 ** 2

    tr_min_idx = np.unravel_index(JR.argmin(), JR.shape)
    t1r_min, t2r_min = t1[tr_min_idx], t2[tr_min_idx]

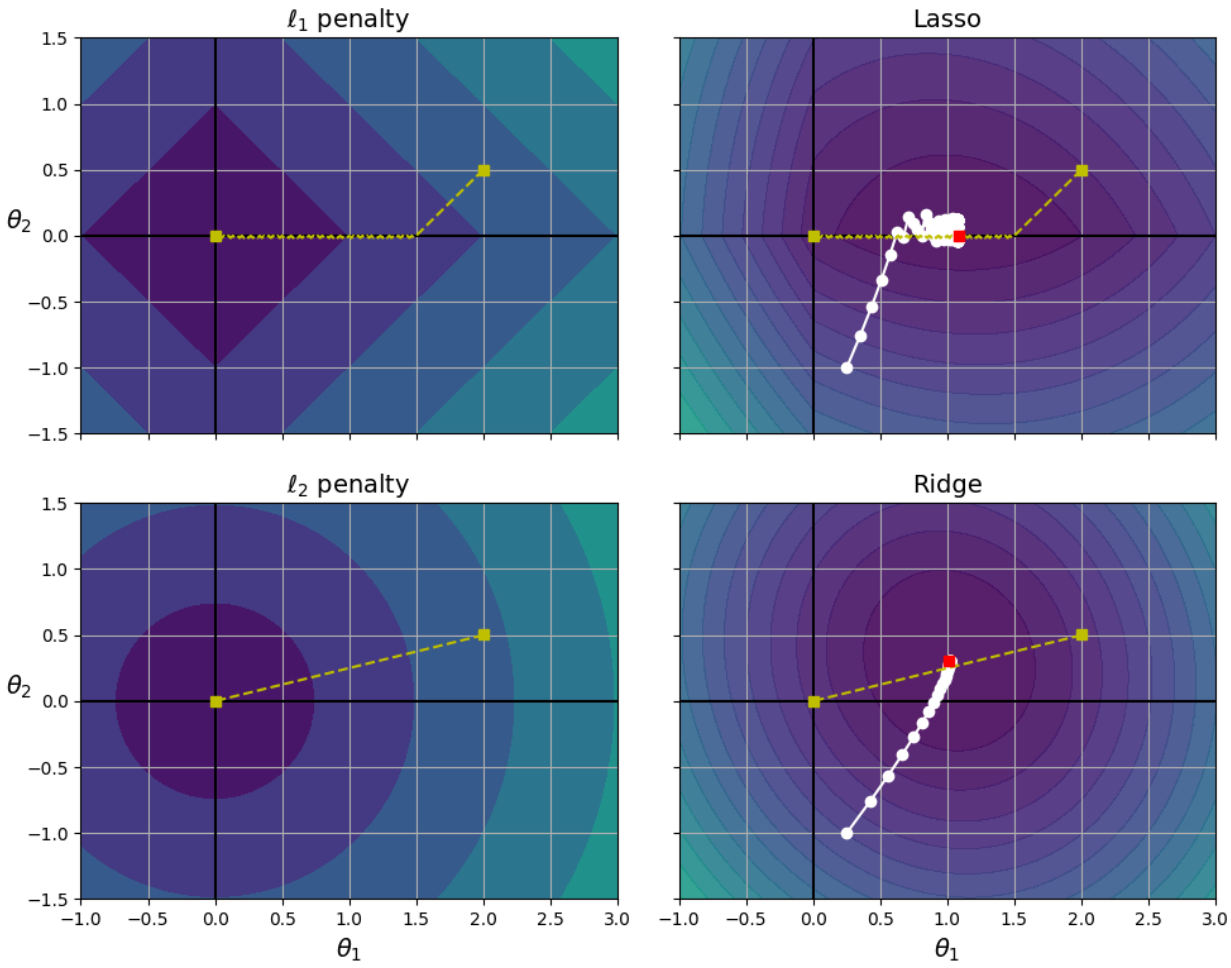
    levels = np.exp(np.linspace(0, 1, 20)) - 1
    levelsJ = levels * (J.max() - J.min()) + J.min()
    levelsJR = levels * (JR.max() - JR.min()) + JR.min()
    levelsN = np.linspace(0, N.max(), 10)

    path_J = bgd_path(t_init, Xr, yr, l1=0, l2=0)
    path_JR = bgd_path(t_init, Xr, yr, l1, l2)
    path_N = bgd_path(theta=np.array([[2.0], [0.5]]), X=Xr, y=yr,
                      l1=np.sign(l1) / 3, l2=np.sign(l2), core=0)
    ax = axes[i, 0]
    ax.grid()
    ax.axhline(y=0, color="k")
    ax.axvline(x=0, color="k")
    ax.contourf(t1, t2, N / 2.0, levels=levelsN)
    ax.plot(path_N[:, 0], path_N[:, 1], "y--")
    ax.plot(0, 0, "ys")
    ax.plot(t1_min, t2_min, "ys")
    ax.set_title(fr"$\ell_{i + 1}$ penalty")
    ax.axis([t1a, t1b, t2a, t2b])
    if i == 1:
        ax.set_xlabel(r"$\theta_1$")
    ax.set_ylabel(r"$\theta_2$", rotation=0)

    ax = axes[i, 1]
    ax.grid()
    ax.axhline(y=0, color="k")
    ax.axvline(x=0, color="k")
    ax.contourf(t1, t2, JR, levels=levelsJR, alpha=0.9)
    ax.plot(path_JR[:, 0], path_JR[:, 1], "w-o")
    ax.plot(path_N[:, 0], path_N[:, 1], "y--")
    ax.plot(0, 0, "ys")
    ax.plot(t1_min, t2_min, "ys")
    ax.plot(t1r_min, t2r_min, "rs")
    ax.set_title(title)
    ax.axis([t1a, t1b, t2a, t2b])
    if i == 1:
```

```
        ax.set_xlabel(r"$\theta_1$")

save_fig("lasso_vs_ridge_plot")
plt.show()
```



Using the figure above, compare the Lasso Regression and the Ridge Regression. What kind of performance is observable from the graph?

- The Ridge Regression is effective at reducing the magnitude of coefficients, which helps in preventing overfitting and stabilizing the model. The Lasso Regression shrinks and does feature selection to the coefficients. This makes it particularly useful in making a simpler and more interpretable model.

3.6 Early Stopping

A very different way to regularize iterative learning algorithms such as Gradient Descent is to stop training as soon as the validation error reaches a minimum. This is called early stopping.

Let's go back to the quadratic dataset we used earlier:

```
from copy import deepcopy
from sklearn.metrics import mean_squared_error
from sklearn.preprocessing import StandardScaler

# extra code – creates the same quadratic dataset as earlier and
splits it
np.random.seed(42)
m = 100
X = 6 * np.random.rand(m, 1) - 3
y = 0.5 * X ** 2 + X + 2 + np.random.randn(m, 1)
X_train, y_train = X[: m // 2], y[: m // 2, 0]
X_valid, y_valid = X[m // 2 :], y[m // 2 :, 0]

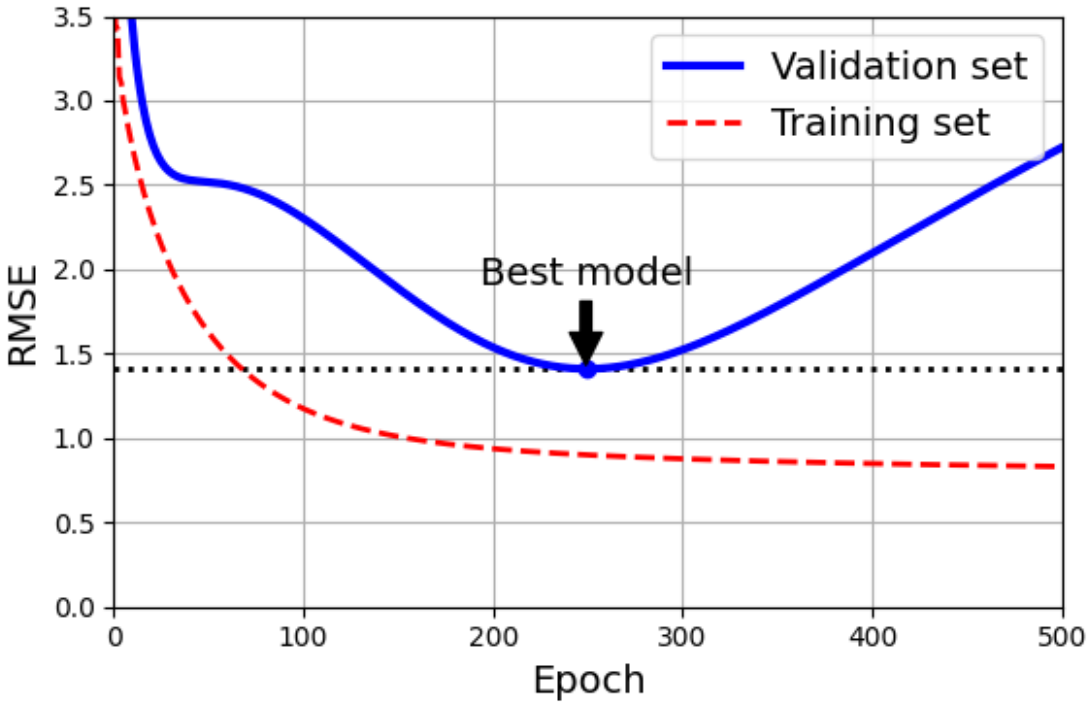
preprocessing = make_pipeline(PolynomialFeatures(degree=90,
include_bias=False),
                              StandardScaler())
X_train_prep = preprocessing.fit_transform(X_train)
X_valid_prep = preprocessing.transform(X_valid)
sgd_reg = SGDRegressor(penalty=None, eta0=0.002, random_state=42)
n_epochs = 500
best_valid_rmse = float('inf')
train_errors, val_errors = [], []  # extra code – it's for the figure
below

for epoch in range(n_epochs):
    sgd_reg.partial_fit(X_train_prep, y_train)
    y_valid_predict = sgd_reg.predict(X_valid_prep)
    val_error = np.sqrt(mean_squared_error(y_valid, y_valid_predict))
    if val_error < best_valid_rmse:
        best_valid_rmse = val_error
        best_model = deepcopy(sgd_reg)

    # extra code – we evaluate the train error and save it for the
figure
    y_train_predict = sgd_reg.predict(X_train_prep)
    train_error = np.sqrt(mean_squared_error(y_train,
y_train_predict))
    val_errors.append(val_error)
    train_errors.append(train_error)

# extra code – this section generates and saves Figure 4–20
best_epoch = np.argmin(val_errors)
plt.figure(figsize=(6, 4))
plt.annotate('Best model',
             xy=(best_epoch, best_valid_rmse),
             xytext=(best_epoch, best_valid_rmse + 0.5),
             ha="center",
             arrowprops=dict(facecolor='black', shrink=0.05))
plt.plot([0, n_epochs], [best_valid_rmse, best_valid_rmse], "k:",
linewidth=2)
```

```
plt.plot(val_errors, "b-", linewidth=3, label="Validation set")
plt.plot(best_epoch, best_valid_rmse, "bo")
plt.plot(train_errors, "r--", linewidth=2, label="Training set")
plt.legend(loc="upper right")
plt.xlabel("Epoch")
plt.ylabel("RMSE")
plt.axis([0, n_epochs, 0, 3.5])
plt.grid()
save_fig("early_stopping_plot")
plt.show()
```



Do you foresee any useful benefits of implementing early stopping?

- Yes. I have noticed that Early Stopping enables to interrupt training when the validation error is low, preventing it from overfitting. It is also an effective and light method for model calculation while still executing a simple model selection and generalization.

3.7 Logistic Regression

Logistic Regression (also called Logit Regression) is com- monly used to estimate the probability that an instance belongs to a particular class (e.g., what is the probability that this email is spam?). If the estimated probability is greater than 50%, then the model predicts that the instance belongs to that class (called the positive class, labeled “1”), or else it predicts that it does not (i.e., it belongs to the negative class, labeled “0”). This makes it a binary classifier.

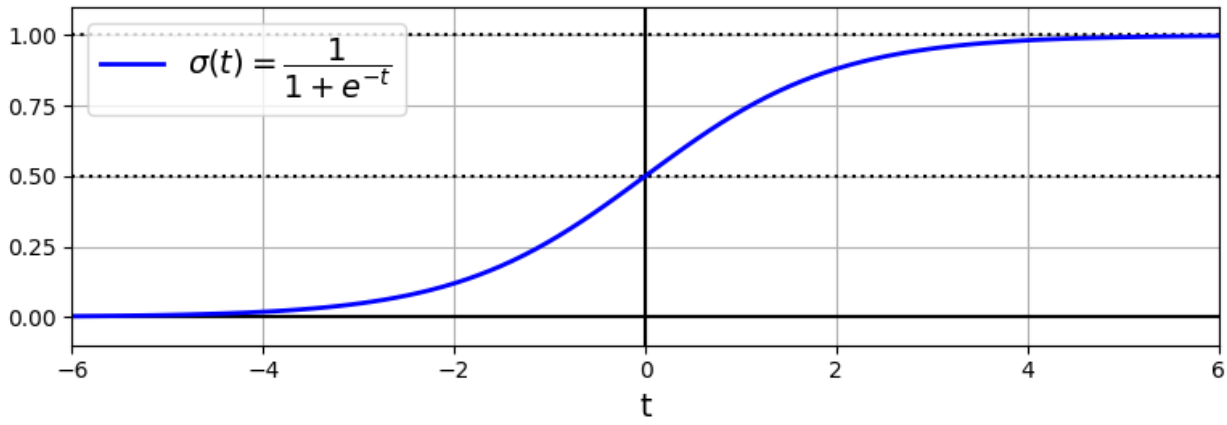
3.7.1 Estimating Probabilities

So how does it work? Just like a Linear Regression model, a Logistic Regression model computes a weighted sum of the input features (plus a bias term), but instead of outputting the result directly like the Linear Regression model does, it outputs the logistic of this result.

```
# extra code – generates and saves Figure 4–21

lim = 6
t = np.linspace(-lim, lim, 100)
sig = 1 / (1 + np.exp(-t))

plt.figure(figsize=(8, 3))
plt.plot([-lim, lim], [0, 0], "k-")
plt.plot([-lim, lim], [0.5, 0.5], "k:")
plt.plot([-lim, lim], [1, 1], "k:")
plt.plot([0, 0], [-1.1, 1.1], "k-")
plt.plot(t, sig, "b-", linewidth=2, label=r"$\sigma(t) = \dfrac{1}{1 +
e^{-t}}$")
plt.xlabel("t")
plt.legend(loc="upper left")
plt.axis([-lim, lim, -0.1, 1.1])
plt.gca().set_yticks([0, 0.25, 0.5, 0.75, 1])
plt.grid()
save_fig("logistic_function_plot")
plt.show()
```



Given the figure above, explain how the equation might be interpreted as the figure and how this provides a logistic regression prediction?

- The logistic function converts a linear combination of features into a probability, which is added with a thresholded and returning a binary classification decision. The curve represents how confident the model is in assigning an instance to the positive class based on the input t.

3.7.2 Decision Boundaries

Let's use the iris dataset to illustrate Logistic Regression. This is a famous dataset that contains the sepal and petal length and width of 150 iris flowers of three different species: Iris-Setosa, Iris-Versicolor, and Iris-Virginica

```
from sklearn.datasets import load_iris

iris = load_iris(as_frame=True)
list(iris)
```

```
['data',
 'target',
 'frame',
 'target_names',
 'DESCR',
 'feature_names',
 'filename',
 'data_module']
```

```
print(iris.DESCR)  # extra code – it's a bit too long
```

```
.. _iris_dataset:

Iris plants dataset
--------------------

**Data Set Characteristics:**

:Number of Instances: 150 (50 in each of three classes)
:Number of Attributes: 4 numeric, predictive attributes and the class
:Attribute Information:
    - sepal length in cm
    - sepal width in cm
    - petal length in cm
    - petal width in cm
    - class:
            - Iris-Setosa
            - Iris-Versicolour
            - Iris-Virginica

:Summary Statistics:

============== ==== ==== ======= ===== ====================
                Min  Max   Mean    SD   Class Correlation
============== ==== ==== ======= ===== ====================
sepal length:   4.3  7.9   5.84   0.83    0.7826
sepal width:    2.0  4.4   3.05   0.43   -0.4194
petal length:   1.0  6.9   3.76   1.76    0.9490  (high!)
petal width:    0.1  2.5   1.20   0.76    0.9565  (high!)
============== ==== ==== ======= ===== ====================
```

```
:Missing Attribute Values: None
:Class Distribution: 33.3% for each of 3 classes.
:Creator: R.A. Fisher
:Donor: Michael Marshall (MARSHALL%PLU@io.arc.nasa.gov)
:Date: July, 1988

The famous Iris database, first used by Sir R.A. Fisher. The dataset is taken
from Fisher's paper. Note that it's the same as in R, but not as in the UCI
Machine Learning Repository, which has two wrong data points.

This is perhaps the best known database to be found in the
pattern recognition literature.  Fisher's paper is a classic in the field and
is referenced frequently to this day.  (See Duda & Hart, for example.) The
data set contains 3 classes of 50 instances each, where each class refers to a
type of iris plant.  One class is linearly separable from the other 2; the
latter are NOT linearly separable from each other.

.. dropdown:: References

  - Fisher, R.A. "The use of multiple measurements in taxonomic problems"
    Annual Eugenics, 7, Part II, 179-188 (1936); also in "Contributions to
    Mathematical Statistics" (John Wiley, NY, 1950).
  - Duda, R.O., & Hart, P.E. (1973) Pattern Classification and Scene Analysis.
    (Q327.D83) John Wiley & Sons.  ISBN 0-471-22361-1.  See page 218.
  - Dasarathy, B.V. (1980) "Nosing Around the Neighborhood: A New System
    Structure and Classification Rule for Recognition in Partially Exposed
    Environments".  IEEE Transactions on Pattern Analysis and Machine
    Intelligence, Vol. PAMI-2, No. 1, 67-71.
  - Gates, G.W. (1972) "The Reduced Nearest Neighbor Rule".  IEEE Transactions
    on Information Theory, May 1972, 431-433.
  - See also: 1988 MLC Proceedings, 54-64.  Cheeseman et al"s AUTOCLASS II
    conceptual clustering system finds 3 classes in the data.
  - Many, many more ...


iris.data.head(3)
```

{"summary":"{\n \"name\": \"iris\",\n \"rows\": 3,\n \"fields\": [\n   {\n   \"column\": \"sepal length (cm)\",\n   \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 0.19999999999999973,\n   \"min\": 4.7,\n   \"max\": 5.1,\n   \"num_unique_values\": 3,\n   \"samples\": [\n   5.1,\n   4.9,\n   4.7\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"sepal width (cm)\",\n   \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 0.2516611478423583,\n   \"min\": 3.0,\n   \"max\": 3.5,\n   \"num_unique_values\": 3,\n   \"samples\": [\n   3.5,\n   3.0,\n   3.2\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"petal length (cm)\",\n   \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 0.0577350269189625,\n   \"min\": 1.3,\n   \"max\": 1.4,\n   \"num_unique_values\": 2,\n   \"samples\": [\n   1.3,\n   1.4\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"petal width (cm)\",\n   \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 3.3993498887762956e-17,\n   \"min\": 0.2,\n   \"max\": 0.2,\n   \"num_unique_values\": 1,\n   \"samples\": [\n   0.2\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   }\n]\n}","type":"dataframe"}

```
iris.target.head(3)  # note that the instances are not shuffled
```

```
0    0
1    0
2    0
Name: target, dtype: int64
```

```
iris.target_names
```

```
array(['setosa', 'versicolor', 'virginica'], dtype='<U10')
```

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split

X = iris.data[["petal width (cm)"]].values
y = iris.target_names[iris.target] == 'virginica'
X_train, X_test, y_train, y_test = train_test_split(X, y,
random_state=42)

log_reg = LogisticRegression(random_state=42)
log_reg.fit(X_train, y_train)
```

```
LogisticRegression(random_state=42)
```

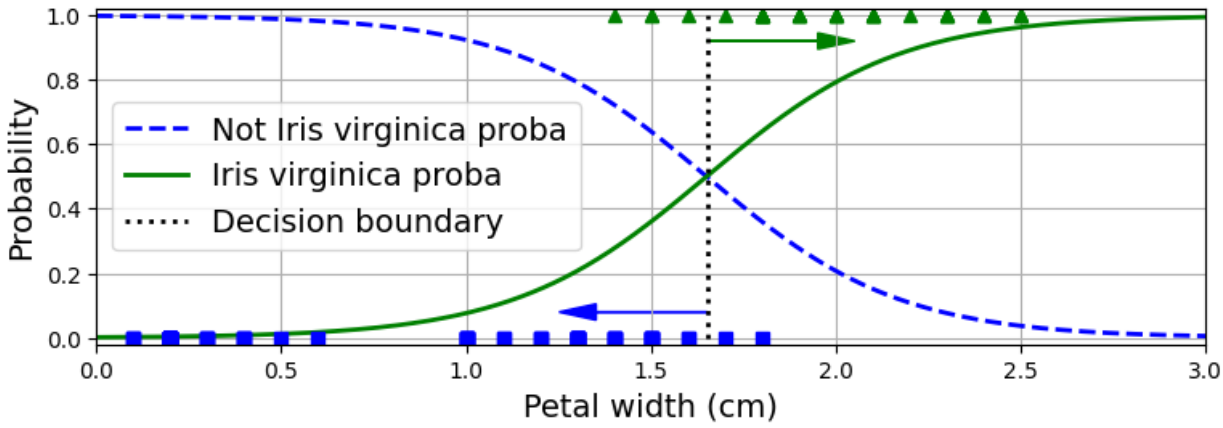
```
X_new = np.linspace(0, 3, 1000).reshape(-1, 1)  # reshape to get a
column vector
```

```
y_proba = log_reg.predict_proba(X_new)
decision_boundary = X_new[y_proba[:, 1] >= 0.5][0, 0]

plt.figure(figsize=(8, 3))  # extra code – not needed, just formatting
plt.plot(X_new, y_proba[:, 0], "b--", linewidth=2,
         label="Not Iris virginica proba")
plt.plot(X_new, y_proba[:, 1], "g-", linewidth=2, label="Iris
virginica proba")
plt.plot([decision_boundary, decision_boundary], [0, 1], "k:",
linewidth=2,
         label="Decision boundary")

# extra code – this section beautifies and saves Figure 4–23
plt.arrow(x=decision_boundary, y=0.08, dx=-0.3, dy=0,
          head_width=0.05, head_length=0.1, fc="b", ec="b")
plt.arrow(x=decision_boundary, y=0.92, dx=0.3, dy=0,
          head_width=0.05, head_length=0.1, fc="g", ec="g")
plt.plot(X_train[y_train == 0], y_train[y_train == 0], "bs")
plt.plot(X_train[y_train == 1], y_train[y_train == 1], "g^")
plt.xlabel("Petal width (cm)")
plt.ylabel("Probability")
plt.legend(loc="center left")
plt.axis([0, 3, -0.02, 1.02])
plt.grid()
save_fig("logistic_regression_plot")

plt.show()
```




```
decision_boundary
```

```
np.float64(1.6516516516516517)
```

```
log_reg.predict([[1.7], [1.5]])
```

```
array([ True, False])
```

```
# extra code – this cell generates and saves Figure 4–24

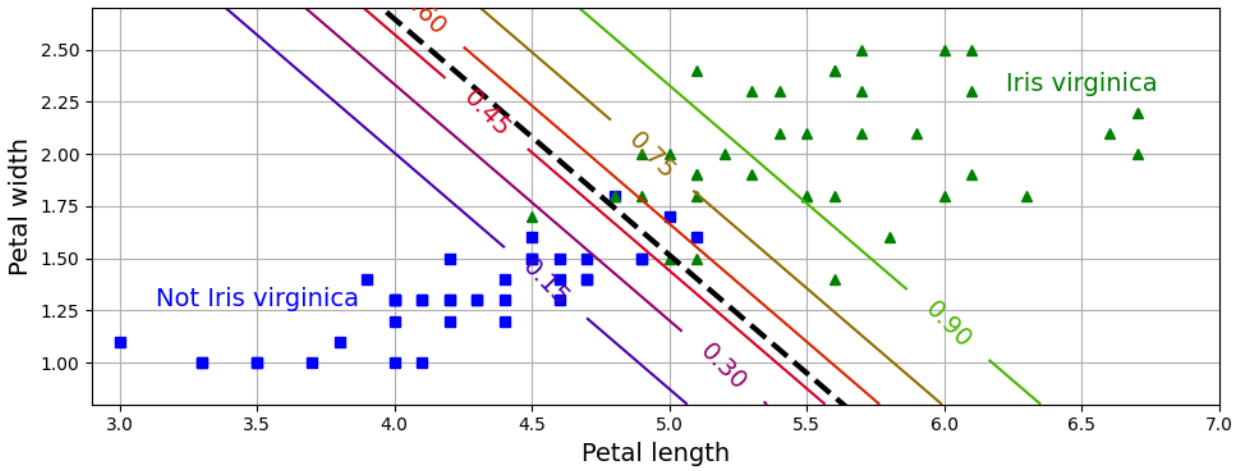
X = iris.data[["petal length (cm)", "petal width (cm)"]].values
y = iris.target_names[iris.target] == 'virginica'
X_train, X_test, y_train, y_test = train_test_split(X, y,
random_state=42)

log_reg = LogisticRegression(C=2, random_state=42)
log_reg.fit(X_train, y_train)

# for the contour plot
x0, x1 = np.meshgrid(np.linspace(2.9, 7, 500).reshape(-1, 1),
                     np.linspace(0.8, 2.7, 200).reshape(-1, 1))
X_new = np.c_[x0.ravel(), x1.ravel()]  # one instance per point on the
figure
y_proba = log_reg.predict_proba(X_new)
zz = y_proba[:, 1].reshape(x0.shape)

# for the decision boundary
left_right = np.array([2.9, 7])
boundary = -((log_reg.coef_[0, 0] * left_right +
log_reg.intercept_[0])
             / log_reg.coef_[0, 1])

plt.figure(figsize=(10, 4))
plt.plot(X_train[y_train == 0, 0], X_train[y_train == 0, 1], "bs")
plt.plot(X_train[y_train == 1, 0], X_train[y_train == 1, 1], "g^")
contour = plt.contour(x0, x1, zz, cmap=plt.cm.brg)
plt.clabel(contour, inline=1)
plt.plot(left_right, boundary, "k--", linewidth=3)
plt.text(3.5, 1.27, "Not Iris virginica", color="b", ha="center")
plt.text(6.5, 2.3, "Iris virginica", color="g", ha="center")
plt.xlabel("Petal length")
plt.ylabel("Petal width")
plt.axis([2.9, 7, 0.8, 2.7])
plt.grid()
save_fig("logistic_regression_contour_plot")
plt.show()
```



L1 and L2 Penalties in Logistic Regression

Logistic Regression, while primarily a classification algorithm, can also be regularized using L1 (Lasso) and L2 (Ridge) penalties, similar to linear regression. These regularization techniques are designed to prevent overfitting by adding a penalty term to the cost function that the model tries to minimize.

1. **L1 Penalty (Lasso Regularization):**
   - **How it works:** The L1 penalty adds the absolute value of the magnitude of the coefficients ($|\theta_i|$) to the cost function. The objective function becomes: `Cost Function + λ * Σ|θi|` (where `λ` is the regularization strength).
   - **Effect:** L1 regularization encourages **sparsity** in the model. This means it tends to drive the weights of less important features exactly to zero, effectively performing **automatic feature selection**. If you have many features, and only a few are truly relevant, L1 can be very useful for simplifying the model.
   - **Application in `LogisticRegression`:** You can specify `penalty='l1'` in `sklearn.linear_model.LogisticRegression`.
2. **L2 Penalty (Ridge Regularization):**
   - **How it works:** The L2 penalty adds the squared magnitude of the coefficients (θ_i^2) to the cost function. The objective function becomes: `Cost Function + λ * Σ(θi^2)`.
   - **Effect:** L2 regularization shrinks the coefficients towards zero but typically does not make them exactly zero. It penalizes large coefficients, encouraging the model to distribute the weight more evenly across all features. This helps to prevent any single feature from dominating the prediction, thereby reducing the model's sensitivity to noise in the training data and improving its generalization.
   - **Application in `LogisticRegression`:** You can specify `penalty='l2'` (which is the default) in `sklearn.linear_model.LogisticRegression`.

The `C` Hyperparameter

In `scikit-learn`'s `LogisticRegression`, instead of a regularization strength `λ` (alpha in Ridge/Lasso), there is a hyperparameter `C`. This `C` is the **inverse of the regularization strength**:

- **Smaller `C` values** correspond to **stronger regularization** (similar to a higher λ). This makes the model simpler and less prone to overfitting.
- **Larger `C` values** correspond to **weaker regularization** (similar to a lower λ). This allows the model to fit the training data more closely, but increases the risk of overfitting.

For example, in the code provided in cell `Z_ObLtar8T3b` for a binary classification, `log_reg = LogisticRegression(random_state=42)` uses the default L2 penalty. In cell `avacyXO68T3c` for a multi-class logistic regression, `log_reg = LogisticRegression(C=2, random_state=42)` applies an L2 penalty with a `C` value of 2, meaning a moderate regularization strength (a smaller `C` would increase the regularization).

Given the above code and the figure provided, how do l1 and l2 penalties apply to the Logistic regression model?

- The L1 penalties enabled automatic feature selection where only few of the features become relevant, effectively simplifying the model. The L2 penalties shrinks large coefficients to distribute the weight more evenly, improving its generalization.

3.7.3 Softmax Regression

The Logistic Regression model can be generalized to support multiple classes directly, without having to train and combine multiple binary classifiers. This is called Softmax Regression, or Multinomial Logistic Regression.

The idea is quite simple: when given an instance x, the Softmax Regression model first computes a score sk(x) for each class k, then estimates the probability of each class by applying the softmax function (also called the normalized exponential) to the scores. The equation to compute sk(x) should look familiar, as it is just like the equation for Linear Regression prediction

```
X = iris.data[["petal length (cm)", "petal width (cm)"]].values
y = iris["target"]
X_train, X_test, y_train, y_test = train_test_split(X, y,
random_state=42)

softmax_reg = LogisticRegression(C=30, random_state=42)
softmax_reg.fit(X_train, y_train)
```

```
LogisticRegression(C=30, random_state=42)
```

```
softmax_reg.predict([[5, 2]])
```

```
array([2])
```

```
softmax_reg.predict_proba([[5, 2]]).round(2)
```

```
array([[0.  , 0.04, 0.96]])
```

```
# extra code – this cell generates and saves Figure 4–25

from matplotlib.colors import ListedColormap

custom_cmap = ListedColormap(["#fafab0", "#9898ff", "#a0faa0"])
```

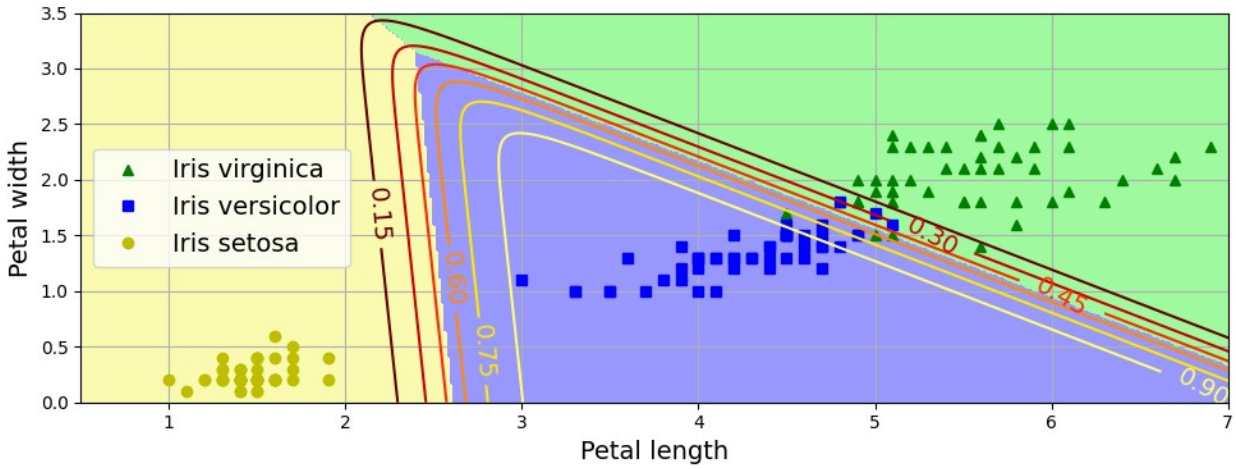
```
x0, x1 = np.meshgrid(np.linspace(0, 8, 500).reshape(-1, 1),
                     np.linspace(0, 3.5, 200).reshape(-1, 1))
X_new = np.c_[x0.ravel(), x1.ravel()]

y_proba = softmax_reg.predict_proba(X_new)
y_predict = softmax_reg.predict(X_new)

zz1 = y_proba[:, 1].reshape(x0.shape)
zz = y_predict.reshape(x0.shape)

plt.figure(figsize=(10, 4))
plt.plot(X[y == 2, 0], X[y == 2, 1], "g^", label="Iris virginica")
plt.plot(X[y == 1, 0], X[y == 1, 1], "bs", label="Iris versicolor")
plt.plot(X[y == 0, 0], X[y == 0, 1], "yo", label="Iris setosa")

plt.contourf(x0, x1, zz, cmap=custom_cmap)
contour = plt.contour(x0, x1, zz1, cmap="hot")
plt.clabel(contour, inline=1)
plt.xlabel("Petal length")
plt.ylabel("Petal width")
plt.legend(loc="center left")
plt.axis([0.5, 7, 0, 3.5])
plt.grid()
save_fig("softmax_regression_contour_plot")
plt.show()
```



Given the code above as an example, how many classes can the Softmax Regression predict? Can it be used to on non-mutually exclusive classes (such as to recognize different faces in a picture)?

4. Supplementary Activity

4.1 Linear Regression

- Choose your own dataset
- Import the dataset
- Determine the number of datapoints, columns and data types
- Remove unneccesary columns
- Do data cleaning such as removing empty values.
- Perform descriptive statistics such as mean, median and mode
- Compute the correlation
- Use One-hot encoding in categorical features
- Apply feature scaling
- Perform linear regression using one independent variable only using numpy
- Perform linear regression using multiple independent variable using statsmodel
- Perform linear regression using sklearn

```
# Import chosen dataset
import pandas as pd
df = pd.read_csv("/content/2023_nba_player_stats.csv")
df.head()
```

```
{"type":"dataframe","variable_name":"df"}
```

```
# Number of datapoints, columns
row, col = df.shape
print(f"Datapoints: {row} / Columns: {col}")
```

```
Datapoints: 539 / Columns: 30
```

```
# Data types
df.dtypes
```

```
PName     object
POS       object
Team      object
Age        int64
GP         int64
W          int64
L          int64
Min      float64
PTS        int64
FGM        int64
FGA        int64
FG%      float64
3PM        int64
3PA        int64
3P%      float64
FTM        int64
FTA        int64
```

```
FT%      float64
OREB       int64
DREB       int64
REB        int64
AST        int64
TOV        int64
STL        int64
BLK        int64
PF         int64
FP         int64
DD2        int64
TD3        int64
+/-        int64
dtype: object
```

```
# Remove unnecessary columns
df.drop(columns="FP")
```

```
{"type":"dataframe"}
```

```
# Remove rows with missing data
df = df.dropna()
df.isnull().sum()
```

```
PName    0
POS      0
Team     0
Age      0
GP       0
W        0
L        0
Min      0
PTS      0
FGM      0
FGA      0
FG%      0
3PM      0
3PA      0
3P%      0
FTM      0
FTA      0
FT%      0
OREB     0
DREB     0
REB      0
AST      0
TOV      0
STL      0
BLK      0
PF       0
```

```
FP       0
DD2      0
TD3      0
+/-      0
dtype: int64
```

```
# Mean
df.select_dtypes(include=['number']).mean()
```

```
Age       25.981273
GP        48.473783
W         24.234082
L         24.239700
Min     1113.872097
PTS      528.327715
FGM      193.370787
FGA      406.762172
FG%       46.758989
3PM       56.852060
3PA      157.602996
3P%       31.825843
FTM       84.734082
FTA      108.389513
FT%       72.667603
OREB      48.061798
DREB     152.028090
REB      200.089888
AST      116.621723
TOV       61.870787
STL       33.576779
BLK       21.440075
PF        92.026217
FP      1046.601124
DD2        4.048689
TD3        0.222846
+/-        0.014981
dtype: float64
```

```
# Median
df.select_dtypes(include=['number']).median()
```

```
Age      25.00
GP       55.00
W        25.00
L        25.50
Min     984.05
PTS     381.00
FGM     140.00
FGA     305.50
FG%      45.55
```

```
3PM      37.00
3PA     109.50
3P%      34.30
FTM      45.00
FTA      61.00
FT%      76.40
OREB     33.00
DREB    120.50
REB     161.00
AST      69.00
TOV      44.50
STL      28.00
BLK      13.00
PF       87.50
FP      833.50
DD2       0.00
TD3       0.00
+/-      -7.50
dtype: float64
```

```
# Mode
df.select_dtypes(include=['number']).mode()
```

```
{"type":"dataframe"}
```

```
# Compute correlation
df.select_dtypes(include=['number']).corr()
```

```
{"type":"dataframe"}
```

```
# No need for One hot encoding, because all columns are numerical data

# Feature scaling
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
scaled_df = scaler.fit_transform(df.select_dtypes(include=['number']))
scaled_df = pd.DataFrame(scaled_df,
columns=df.select_dtypes(include=['number']).columns)
scaled_df.head()
```

```
{"type":"dataframe","variable_name":"scaled_df"}
```

```
# Select independent and dependent variables from the scaled dataframe
X_single = scaled_df[['Min']].values  # Independent variable
y_single = scaled_df[['PTS']].values  # Dependent variable

# Add a bias term to
X_single_b = np.c_[np.ones((X_single.shape[0], 1)), X_single]

# Calculate theta using the Normal Equation
```

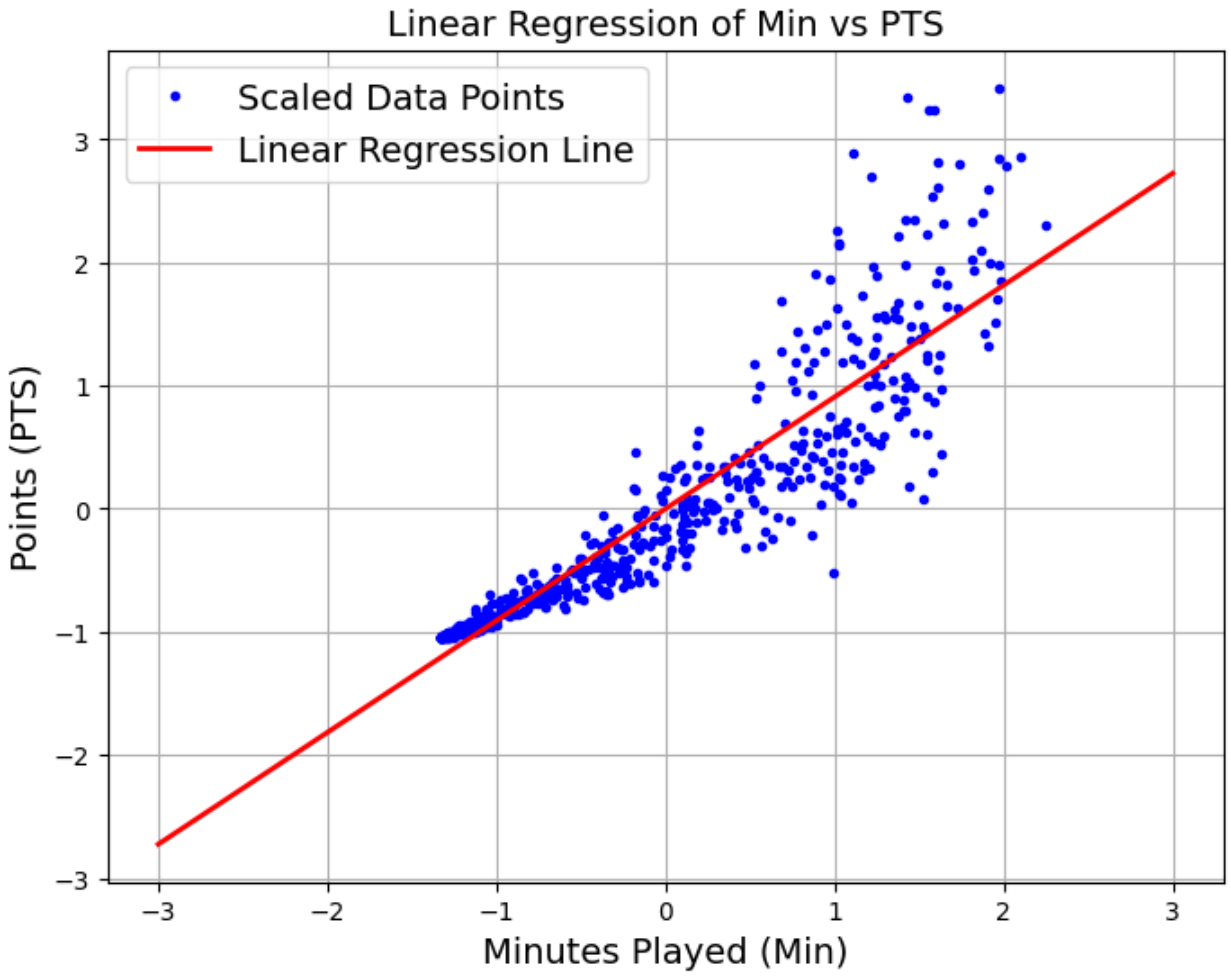
```
theta_single_numpy = np.linalg.inv(X_single_b.T @ X_single_b) @
X_single_b.T @ y_single

print("Intercept (theta_0):", theta_single_numpy[0][0])
print("Coefficient for Min (theta_1):", theta_single_numpy[1][0])

# Make predictions
X_new_single = np.array([[-3], [3]]) # Range of scaled 'Min' values
for prediction
X_new_single_b = np.c_[np.ones((2, 1)), X_new_single]
y_predict_single = X_new_single_b @ theta_single_numpy

# Plot the results
plt.figure(figsize=(8, 6))
plt.plot(X_single, y_single, "b.", label="Scaled Data Points")
plt.plot(X_new_single, y_predict_single, "r-", linewidth=2,
label="Linear Regression Line")
plt.xlabel("Minutes Played (Min)")
plt.ylabel("Points (PTS)")
plt.title("Linear Regression of Min vs PTS")
plt.legend()
plt.grid(True)
plt.show()
```

```
Intercept (theta_0): -1.986258379993444e-16
Coefficient for Min (theta_1): 0.9080746413125065
```




```
# Perform linear regression using multiple independent variable using statsmodel

# Perform linear regression using sklearn
```

4.2 Polynomial Regression

- Choose your own dataset
- Import the dataset
- Perform polynomial regression using sklearn and polyfit
- Measure the performance for each polynomial degree.
- Plot the performance of the model for each polynomial degree.

4.3 Logistic Regression

- Choose your own dataset
- Import the dataset
- Determine the number of datapoints, columns and data types
- Remove unneccesary columns
- Do data cleaning such as removing empty values(NaN), replacing missing data .

- Perform descriptive statistics such as mean, median and mode
- Perform data visualization
- Solve classification problem using Logistic Regression
- Evaluate the model using classification report, accuracy and confusion matrix

5. Summary, Conclusions and Lessons Learned

Proprietary Clause

Property of the Technological Institute of the Philippines (T.I.P.). No part of the materials made and uploaded in this learning management system by T.I.P. may be copied, photographed, printed, reproduced, shared, transmitted, translated, or reduced to any electronic medium or machine-readable form, in whole or in part, without the prior consent of T.I.P.

Disclaimer

Contents of this Notebook are based on Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow 3rd Edition by Aurelien Geron. A copy of the complete text can be obtained here: https://www.oreilly.com/library/view/hands-on-machine-learning/9781492032632/